

An imperative core

Von Neumann programming languages use variables to imitate the computer's storage cells; control statements elaborate its jump and test instructions; and assignment statements imitate its fetching, storing, and arithmetic. . . Each assignment statement produces a one-word result. The program must cause these statements to be executed many times in order to make the desired overall change in the store, since it must be done one word at a time.

John Backus, *Can Programming Be Liberated from the von Neumann Style?*

In your prior programming experience, you may have used a *procedural* language such as Ada 83, Algol 60, C, Cobol, Fortran, Modula-2, or Pascal. Or you may have used a procedural language extended with object-oriented features, such as Ada 95, C#, C++, Eiffel, Java, Modula-3, Objective C, or Python—although these hybrid languages support an object-oriented style, they are often used procedurally. Procedural programming is a well-developed style with identifiable characteristics:

- Code is organized into *procedures*; a procedure is a sequence of *commands*, each of which tells the computer, “do something!” (“Command” is what we said in the 1960s; these days, we say “statement.”) A command is executed for its *effects* on the state of the machine; procedures continually change the “mutable state” of the machine—the values contained in the various machine words—by assignment.
Each command in a procedure can itself be implemented by another procedure, and so on; procedures can be designed from the top down using *step-wise refinement* (Wirth 1971).
- Data is processed one word at a time; words are commonly organized into arrays or records. An array is typically processed by a loop, and a record is typically processed by a sequence of commands; these control constructs reflect an element-by-element approach to data processing. Loops are typically written using “structured” looping constructs such as *for* and *while*; recursive procedures are not commonly used.
- Both control and data mimic machine architecture. The control constructs *if* and *while* combine conditional and unconditional jump instructions in simple ways; *goto*, when present, exposes the machine's unconditional jump. Arrays and records are implemented by contiguous blocks of memory. Pointers are addresses; assignment is often limited to what can be accomplished by a single load or store instruction.

Mimicking a machine has its advantages: costs can be easy to predict, and a debugger can be built by adapting a machine-level debugger.

1

An imperative core

12

The procedural style can be embodied in a language. In this chapter, that language is *Impcore*. Impcore is named for the standard imperative core—assignments, loops, conditionals, and procedures—that is found in almost all programming languages, including not only “imperative” or “procedural” languages but also many “functional” and “object-oriented” languages. Impcore is best explained by example, so let’s start by defining a global variable *n*, which is initialized to 3:

```
12a. <transcript 12a>≡ 12b>
-> (val n 3)
3
```

The arrow “-> ” is the interpreter’s prompt; text following a prompt is my input; and text on the next line is the interpreter’s response:¹

To continue, let’s define a function, which I’ll name *x-3-plus-1*. It multiplies a number *k* times 3, then adds 1 to the product:

```
12b. <transcript 12a>+≡ <12a 12c>
-> (define x-3-plus-1 (k)
    (+ (* 3 k) 1))    ;; returns 3 * k + 1
```

The syntax may look different from what you are used to, including the dashes in the function’s name, but before we dive into the differences, let’s compute with the variable *n* and function *x-3-plus-1* that I just defined. I use them in a loop that tries to reduce *n* to 1 by halving *n* when it is even and replacing it with $3n + 1$ when it is odd. (This loop is believed to terminate for any positive *n*, but at press time, no proof is known.) In the loop, each line of Impcore code is commented with analogous C code on the right; if you know C, C++, Java, JavaScript, or something similar, the comments should help you interpret Impcore’s syntax.

```
12c. <transcript 12a>+≡ <12b 21a>
-> (begin                                ;; {
    (while (> n 1)                        ;; while (n > 1)
      (begin                             ;; {
        (println n)                     ;; printf("%d\n", n);
        (if (= (mod n 2) 0)              ;; if (n % 2 == 0)
          (set n (/ n 2))                ;; n = n / 2;
          (set n (x-3-plus-1 n))))       ;; else n = x_3_plus_1(n); }
    n)                                  ;; return n; }

3
10
5
16
8
4
2
1
```

In these examples, you might notice that

- Where C uses curly brackets, Impcore uses *begin*.
- Impcore doesn’t use *else* or *return* keywords.
- Impcore uses a lot of ugly parentheses, but no commas.
- Impcore puts operators like *+*, ***, *>*, *=*, and */* in places that might look strange.

¹Most of the examples in this book are written using the Noweb system for *literate programming*. Bold labels such as **12a** identify chunks of examples or code, and pointers such as “12b>” point to subsequent chunks. Noweb is explained more fully in Section 1.6 on page 39.

Impcore, like every other language in this book, uses *fully parenthesized, prefix* syntax, in which each operator precedes its arguments. C uses *infix* syntax; each binary operator appears between its arguments. In C's infix syntax, order of evaluation is determined by "operator precedence," which is fine for stuff you use every day—but not so good when you have to figure out whether & and && have the same precedence and where they both stand with respect to |. In Impcore's prefix syntax, order of evaluation is determined by parentheses; it might be ugly, but you don't have to remember any operator precedence. Prefix parenthesized syntax, which is used in Scheme, Common Lisp, Emacs Lisp, Racket, Clojure, and the many other languages of the Lisp family, puts every operator and every programmer on the same footing—the parentheses eliminate any possible ambiguity.

These examples show Impcore, which is going to help us look at programming languages in a deep, systematic way. But first, let's get the big picture of what aspects we will look at.

1.1 LOOKING AT LANGUAGES

A program is ultimately formed from the individual characters of its code. And before code runs, it passes through several different phases, each of which is governed by its own rules. Phases and their rules can be hard to identify, but for any programming language, three sets of rules are essential. They tell us

- How code is formed
- What checking it undergoes before it is deemed OK to run
- What happens when it runs

To highlight these rules, let's look at some code that breaks them. Because some of the rules involve type checking, we'll look at some C code. (If C is not so familiar, try thinking about Java, which has the same kinds of rules.)

Imagine that I have a two-dimensional point on the plane, and that I want to add 3 to its *x* coordinate. In C, a point *p* can be defined like this:

```
struct { int x; int y; } p;
```

To add 3 to *p*'s *x* coordinate, I write an assignment statement. But if a letter *x* is deleted by a cat that walks across my keyboard, the cat might leave this code:

```
p. = p.x + 3;
```

This code is rejected by the C compiler, which reports a *syntax error*. The compiler might flag the = sign after the dot, where it would prefer a name. The code is *ill formed*.

I know I want the = sign, and if I'm programming after midnight, I might just remove the dot:

```
p = p.x + 3;
```

This code is well formed, but a C compiler will report a *type error*: *p* is not the type of thing you can assign a number to. The code is *ill typed*.

The code I meant to write is

```
p.x = p.x + 3;
```

This code is well formed and well typed, and the compiler is happy. But a happy

compiler doesn't guarantee a happy program. Suppose I write

```
int n = 0;  
p.x = p.x / n;
```

1

An imperative core

14

This code is also well formed and well typed, so it makes the compiler happy, but when it runs, nothing good will happen. The best I can hope for is that my operating system will report a *run-time error*, maybe a “floating-point exception.” (The example exhibits what the C standard calls “undefined behavior,” which the system is not obligated to report.) The code is *ill behaved*.

Useful code is well formed, well typed, and well behaved. Learning precisely what that means, for several different languages, is half of this book. (The other half is learning how to use the languages effectively.) The key concepts are as follows:

- Code is formed according to two sets of rules: Characters are clumped into groups called *tokens* according to *lexical* rules, and tokens are grouped into definitions, statements, and so on according to *syntactic* rules. The syntactic rules are the important ones. Syntax is so important that we talk about two varieties: *concrete syntax*, which is how we write the code, and *abstract syntax*, which is how we think about the code's structure. Impcore's concrete and abstract syntax are presented in Sections 1.2 and 1.3, respectively.
- Although there are myriad ways that code can be checked before it is deemed OK to run, the most practical method is *type checking*, which follows the rules of a *static type system*. Impcore does not have a static type system; type checking and its companion, *type inference*, are not explored in depth until Chapters 6 and 7.
- The behavior of running code is specified by its *semantics* (sometimes called *dynamic semantics*). Semantics can be written in several different styles, but this book uses *operational semantics*, a style that has dominated the field since the 1990s. Operational semantics has a learning curve, and to help you climb it, each language in this book is implemented by an *interpreter*. The interpreter demonstrates what the language's semantics is trying to tell us, and by enabling us to run code, it also helps us learn to use the language effectively.

The operational semantics of Impcore is presented in Section 1.5. In Section 1.6, the operational semantics is used to guide the construction of Impcore's interpreter, and in Section 1.7, the operational semantics is used to prove properties of Impcore code.

To understand *any* language's operational semantics or type system, you must first know how its programs are formed. To learn, try this approach:

1. Understand the lexical structure, especially of things like comments and string literals.
2. Look for familiar *syntactic categories* (page 16), like definitions, declarations, statements, expressions, and types.
3. In each category, look for familiar *forms*: loops, conditionals, function applications, and so on. To identify what's familiar, mentally translate concrete syntax into abstract syntax.

This approach calls for an understanding of lexical structure, grammars, and the two varieties of syntax.

Lexical structure rarely requires much thought. If you can spot comments, string literals, and token boundaries, you're good to go. For example, in C, you need to see $3*n+1$ as 5 tokens, " $3*n+1$ " as a single token (a string literal), and $/*3*n+1*/$ as no tokens at all (just a comment). In this book's bridge languages, a comment begins with a semicolon, there are no string literals, and token boundaries are found only at brackets or whitespace. For example, in Impcore, $3*n+1$ is a single token (a name).

Grammars

Concrete syntax is specified using a *grammar*, which tells us what sequences of tokens are well formed. For example, the following toy grammar shows four ways to form an expression *exp*:

$$exp ::= variable\text{-}name \mid numeral \mid exp + exp \mid exp * exp.$$

The grammar says that an expression may be a variable, a numeral, the sum of two expressions, or the product of two expressions. Each alternative is a *syntactic form*.

This grammar, like any grammar, can be used to *produce* an expression by replacing *exp* with any of the four alternatives on the right-hand side, and continuing recursively until every *exp* has been replaced with a variable or a numeral. For example, the C expression $3 * n + 1$ can be produced in this way.

The toy grammar is *compositional*: big expressions are made by composing smaller ones. Every interesting programming-language grammar supports some kind of composition; even an assembly language allows you to compose long sequences of instructions by concatenating shorter sequences. Compositional syntactic structure is part of what makes something a programming language. More compositional structure can be found in the grammar for Impcore (page 18).

Two varieties of syntax

When we *write concrete* syntax, we should be *thinking* about *abstract* syntax. For example, concrete syntax tells us that the " $3n + 1$ " loop is written using a *while* keyword, and that in C the loop condition goes between round brackets (parentheses). But we should be thinking "while loop with a condition and a body," which is what abstract syntax tells us: it names the form (WHILE) and says that a WHILE loop is formed from an expression (the condition) and a statement (the body). Abstract syntax ignores syntactic markers like keywords and brackets.

Abstract syntax helps us recognize familiar forms even when they are clothed in unfamiliar concrete syntax. For example, you can probably recognize loops written in C, Icon, Impcore, Python, Modula-3, Scala, and Standard ML:

```
while (n > 1) n = n / 2;
while n > 1 do n := n / 2
(while (> n 1) (set n (/ n 2))
while n > 1: n = n / 2
WHILE n > 1 DO n := n / 2 END
while (n > 1) { n = n / 2; }
while !n > 1 do n := !n div 2
```

Each language uses a different concrete syntax, but abstract syntax provides a kind of X-ray vision: under their clothes, all these loops are the same.

Abstract syntax is not just a tool for thought. It also gives us *abstract-syntax trees*, the data structure used to represent code in most compilers and interpreters.

In this book, abstract-syntax trees appear in C code (Chapters 1 to 3) and in Standard ML code (Chapters 5 to 10). Abstract syntax also provides a compact notation for operational semantics and type systems. Using such a notation, all the `while` loops above would be written the same way: `WHILE(e, s)`, where *e* stands for the condition and *s* stands for the body.

Familiar syntactic categories and forms

Abstract syntax is rarely specified explicitly; you'll almost always infer it by reading a grammar that describes concrete syntax. A real grammar will have many left-hand sides like *exp*; these symbols are called *nonterminal symbols*, or just *nonterminals*. Sometimes the nonterminals tell you exactly what the important phrases in a language are, but if the grammar has been engineered primarily to help a compiler to convert its input into an abstract-syntax tree, many nonterminals will be annoying or distracting. For example, a nonterminal like *explist1* (a list of one or more expressions separated by commas) adds nothing to our understanding.

To understand the structure of a grammar, search the nonterminals for *syntactic categories*. A syntactic category is a group of syntactic forms that share an important role; for example, the role of an expression is to be evaluated to produce a value (and possibly also have a *side effect*). Typically, two phrases in the same syntactic category can be interchanged without affecting the well-formedness of a program. For example, any of these expressions could be used on the right-hand side of the assignment to `p.x`:

`p.x + 3` `p.x / n` `p->next` `2 * p`

Assigning any of these expressions to `p.x` would be well formed, but as shown above, the division `p.x / n` might not be well behaved, and the assignments of `p->next` and `2 * p` aren't well typed.

Syntactic categories aren't arbitrary, and in any programming language, there are at least four common categories worth looking for:

- A *definition* introduces a new thing and gives it a name. Forms to look for include forms that define functions, variables, and maybe types; Impcore marks its function-definition and variable-definition forms with keywords `define` and `val`.
- A *declaration* introduces a new thing, with a name, but doesn't yet define the thing—instead, a declaration promises that the thing is defined elsewhere. Because anything that can be declared eventually has to be defined, the forms to look for are forms for declaring things that can be defined. In C and C++, declaration forms are mostly found in `.h` files. Impcore has no declaration forms; in this book, declaration forms aren't used until Chapter 9.
- An *expression* is evaluated to produce a value, and possibly also have a side effect. Forms to look for include variables, literal values, function applications, maybe infix operators, and hopefully a conditional form (like C's ternary expression $e_1 ? e_2 : e_3$). Impcore has all these forms except infix operators.
- A *statement* is executed for side effect; it doesn't produce a value. Side effects might include printing, changing the value of a variable, or changing some value in memory, among others. Forms to look for include loops (`while`, `for`), conditionals (`if`), sequencing (`begin`), and if you're lucky, a case or `switch` statement. Impcore doesn't actually have statements; its `while` and `if` forms are expressions.

Impcore's lack of statements might surprise you, but it's a well-known design choice. Providing loops and conditionals as expressions, not statements, makes a language *expression-oriented*. All functional languages are expression-oriented; among procedural languages, Icon is expression-oriented (Griswold and Griswold 1996); and among languages with object-oriented features, Scala is expression-oriented (Odersky, Spoon, and Venners 2019). Making a language expression-oriented simplifies the syntax a bit; for example, an expression-oriented language needs only one conditional form, whereas a language like C has both a conditional statement and a conditional expression.

With your eye out for familiar definition and expression forms, you're ready to be fully introduced to Impcore.

1.2 THE IMPCORE LANGUAGE

An Impcore program is a sequence of *definitions*, each of which contains at least one *expression*. Definitions come in two main forms: a definition of a variable, such as `(val n 5)`, and a definition of a function, such as `(define double (x) (+ x x))`. Impcore is implemented by an interactive interpreter, and because it's wonderfully convenient to be able to type in an expression, have it evaluated, and see the result, Impcore also counts an expression, such as `(+ 2 2)`, as a “definition”—in this case, a definition of the global variable `it`. The global variable `it` thereby “remembers the last expression typed in.” (The variable `it` has played this special role in interactive interpreters for over thirty years.)

With the big picture of definitions and expressions in mind, we're ready to look at the complete lexical and syntactic structure of Impcore—how programs are formed—and some more examples.

1.2.1 Lexical structure and concrete syntax

Impcore fits the model described on page 14: the rules for forming programs are divided into lexical rules and syntactic rules. The lexical rules, with only minor variations, are the same for all the languages in the book:

- A semicolon starts a comment, which runs to the end of the line on which it appears.
- Each bracket character—`(`, `)`, `[`, `]`, `{`, or `}`—is a token by itself. Impcore uses only the round and square brackets, not the curly ones.
- Other characters are clumped into tokens that are as long as possible; a token ends only at a bracket, a semicolon, or whitespace.
- Aside from its role in delimiting tokens, whitespace is ignored.

If you're used to C or Java, these rules may surprise you in a one small way: inputs like `x+y` and `3rd` are *single* tokens—in Impcore, each is a valid name!

In this book, syntactic rules—that is, grammars—are written using Extended Backus-Naur Form, usually abbreviated as EBNF. EBNF is based on plain BNF, which is ubiquitous. BNF has been extended in many different ways, and the EBNF I use is best understood through an example: the grammar for Impcore, which appears in Figure 1.1 on the next page. (ENBF is explained more fully in Appendix A.)

Figure 1.1, like any other grammar, lists nonterminal symbols like *def*, *unit-test*, *exp*, and so on. Each nonterminal is followed by the `::=` symbol (pronounced “produces”), followed by the forms of the phrases that the nonterminal can produce. Alternative forms are separated by vertical bars, as `(one thing | another)`. In each

```

def ::= (val variable-name exp)
      | exp
      | (define function-name (formals) exp)
      | (use file-name)
      | unit-test

unit-test ::= (check-expect exp exp)
              | (check-assert exp)
              | (check-error exp)

exp ::= literal
      | variable-name
      | (set variable-name exp)
      | (if exp exp exp)
      | (while exp exp)
      | (begin {exp})
      | (function-name {exp})

formals ::= {variable-name}

literal ::= numeral

numeral ::= token composed only of digits, possibly prefixed with a plus
              or minus sign

any *-name ::= token that is not a bracket, a numeral, or one of the “re-
              served” words shown in typewriter font

```

Any syntactic form that is written with a matching pair of round brackets ($\dots$) may equally well be written with a matching pair of square brackets [$\dots$]. They mean the same.

Figure 1.1: Concrete syntax of Impcore

syntactic form, a token that is supposed to appear literally (like `val` or `while`) is written in typewriter font; a name that stands for a token or for a sequence of tokens (like *def*, *variable-name*, or *exp*) is written in *italic font*. Finally, a phrase that can be repeated is written in curly brackets, like the {*exp*} in the `begin` form; a `begin` may contain any number of *exps*, including zero.

Figure 1.1 confirms that Impcore’s concrete syntax is fully parenthesized; wherever a sequence of tokens appears, that sequence is wrapped in brackets. You may find this syntax unattractive, especially in complex expressions. But when you’re learning multiple languages, it’s great not to have to worry about operator precedence. And when you must write a deeply nested expression with a ton of brackets, you can reveal its structure by mixing round and square brackets—as long as round matches round and square matches square, the two shapes are interchangeable.

Figure 1.1 begins with definitions. The definition form (val *x* *e*) defines a new global variable *x* and initializes it to the value of the expression *e*. A global variable must be defined before it is used or assigned to. Next, any expression *exp* may be used as a definition form; it defines or assigns to the global variable `it`. And the definition form (define *f* (*x*₁ $\dots$ *x*_{*n*}) *e*) defines a function *f* with formal parameters *x*₁ to *x*_{*n*} and body *e*.

The `val`, *exp*, and `define` forms are what I call *true definitions*; these forms should be thought of as part of a program. The remaining forms, which I call *extended definitions*, are more like instructions to the interpreter. The (use *file-name*)

form tells the interpreter to read and evaluate the definitions in the named file. A check-expect, check-assert, or check-error form tells the interpreter to remember a test and to run it *after* reading the file in which the test appears.

The expression forms, which all appear in the example while expression at the beginning of the chapter, constitute a bare minimum needed for writing imperative or procedural code. The forms can express a literal value, a variable, an assignment (set), a conditional (if), a loop (while), a sequence (begin), and a function application (any other bracketed form).

Variables and functions are named according to liberal rules: almost any non-bracket token can be a name. Only the words val, define, use, check-expect, check-assert, check-error, set, if, while, and begin, are *reserved*—they cannot be used to name functions or variables. And a *numeral* always stands for a number; a numeral cannot be used to name a function or a variable.

When Impcore starts, some names are already defined. These include *primitive* functions +, -, *, /, =, <, >, println, print, and printu, and also *predefined* functions and, or, not, <=, >=, !=, mod, and negated. A set of defined names forms a *basis*; the set of names defined at startup forms the *initial basis*. The concepts of primitive, predefined, and basis are explained in Section 1.2.6 (page 26).

1.2.2 Talking about syntax: Metavariables

Once we know how code is formed, we can talk about what happens when it is run. To talk about *any* well-formed code, not just particular codes, we use names called *metavariables*. In this chapter, the metavariables used to talk about code are as follows:

<i>e</i>	Any expression
<i>d</i>	Any definition
<i>n</i>	Any numeral
<i>x</i>	Any name that is meant to refer to a variable or a parameter
<i>f</i>	Any name that is meant to refer to a function

To talk about more than one expression or name, we use subscripts. For example, we write an if expression as (if e_1 e_2 e_3). Because each subexpression might be different from the other two, each one is referred to by its own metavariable.

Metavariables are distinct from *program variables*. Program variables *appear* in source code; metavariables *stand for* source code. Metavariables are written in *math italics* and program variables in typewriter font. For example, x is a program variable: it is a name that can appear in source code. But *x* is a metavariable: it stands for *any* name that could appear in source code. Metavariable *x* might stand for x, but it might also stand for y, z, i, j, or any other program variable. To illustrate the difference, when we write (val x 3), we mean the definition of global program variable x. But when we write (val *x* *e*), we mean a template that can stand for *any* definition of *any* global variable.

Metavariables differ from program variables in one other crucial respect: a metavariable can't be renamed without changing its meaning. So although you can write (define double (n) (+ n n)) and it means *exactly* the same thing as (define double (x) (+ x x)), you *cannot* write (val *m* *g*) and have it mean the same thing as (val *x* *e*): (val *x* *e*) is a template for an expression, but because *m* and *g* mean nothing when used as metavariables, (val *m* *g*) is gibberish. If you want a distinct name for a new metavariable, the most you can do is decorate the original name in some way—traditionally with a prime or a subscript. For example, (val x' e_4) is also a template for an expression, and it's an expression that might be different from (val *x* *e*).

- To evaluate a literal expression, which in Impcore takes the form of a numeral n , we return the 32-bit integer that n stands for.
- To evaluate an expression that takes the form of a variable name x , we find the variable that x refers to, which must be either a function parameter or a global variable defined with `val`, and we return the value.
- To evaluate an expression of the form `(set  $x$   $e$ )`, we evaluate e , assign its value to the variable x , and return the value. Variable x must be a global variable or a formal parameter.
- To evaluate an expression of the form `(if  $e_1$   $e_2$   $e_3$ )`, we first evaluate e_1 . If the result is nonzero, we evaluate e_2 and return the result; otherwise we evaluate e_3 and return the result.
- To evaluate an expression of the form `(while  $e_1$   $e_2$ )`, we first evaluate e_1 . If the result is not zero, we evaluate e_2 , then start evaluating the loop again with e_1 . We continue until e_1 evaluates to zero. When e_1 evaluates to zero, looping ends, and the result of the while loop is zero. (The result of evaluating a while expression is *always* zero, but because while is typically evaluated for its side effects, we usually don't care.)
- To evaluate an expression of the form `(begin  $e_1 \dots e_n$ )`, we evaluate expressions e_1 through e_n in that order, and we return the value of e_n .
- To evaluate an expression of the form `(f  $e_1 \dots e_n$ )`, we evaluate expressions e_1 through e_n in that order, calling the results $v_1, \dots, v_n$. We then apply function f to $v_1, \dots, v_n$ and return the result. Function f may be *primitive* or user-defined; if f names a user-defined function, we find f 's definition, let the names in the *formals* stand for $v_1, \dots, v_n$, and return the result of evaluating f 's body. If f names a primitive function, we apply it as described Section 1.2.4 (page 23).

Figure 1.2(a): Rules for evaluating expressions

1.2.3 What the syntactic forms do

Metavariables enable us to talk about *any* definition or expression of a certain form. And what we most want to say is *operational*: what happens when a form is evaluated. To explain the evaluation of Impcore's syntactic forms, I want to draw on your experience and intuition, so I use informal English. But informal English lacks precision, so in Section 1.5 (page 29), I also provide a precise, formal semantics.

I start with the most fundamental syntactic category: expressions. When an expression is evaluated, it returns a value. A value is not code, so to refer to a value, we need another metavariable:

v Any value

In Impcore, all values are integers. Evaluating an expression produces a value and may also have a *side effect*; in Impcore, possible side effects include printing something or changing the value of a variable.

A literal 3 evaluates to the value 3.

21a. $\langle \text{transcript 12a} \rangle + \equiv$ $\langle 12c \ 21b \rangle$
-> 3
3

The global variable n defined earlier in the chapter evaluates to its value:

21b. $\langle \text{transcript 12a} \rangle + \equiv$ $\langle 21a \ 21c \rangle$
-> n
1

A set changes a variable's value:

21c. $\langle \text{transcript 12a} \rangle + \equiv$ $\langle 21b \ 21d \rangle$
-> (set n -13)
-13
-> n
-13

A conditional evaluates to a value that depends on the condition. For example, a conditional can compute the absolute value of n:

21d. $\langle \text{transcript 12a} \rangle + \equiv$ $\langle 21c \ 21e \rangle$
-> (if (< n 0) (negated n) n)
13

A loop's value is always 0, but its evaluation can change the values of variables:

21e. $\langle \text{transcript 12a} \rangle + \equiv$ $\langle 21d \ 21f \rangle$
-> (while (< n 0) (set n (+ n 10)))
0
-> n
7

A begin is used to sequence expressions for their side effects, like printing:

21f. $\langle \text{transcript 12a} \rangle + \equiv$ $\langle 21e \ 21g \rangle$
-> (begin (prнту 169) (println 2021) -1)
@2021
-1

Function application may call a primitive, predefined, or user-defined function, like <, negated, +, prнту, println, or x-3-plus-1.

Figure 1.2(b): Examples of evaluating expressions

Expressions are evaluated according to the rules in Figure 1.2(a), and corresponding examples appear in Figure 1.2(b)—except for calls and names, because examples of calls and names won't fit in the figure. To illustrate calls and names, I show what the Impcore interpreter does with two function definitions and calls. After evaluating a function's definition, the interpreter echoes the function's name.

21g. $\langle \text{transcript 12a} \rangle + \equiv$ $\langle 21f \ 22a \rangle$
-> (define add1 (n) (+ n 1))
add1
-> (define double (n) (+ n n))
double
-> (add1 4)
5
-> (double (+ 3 4))
14

negated	27c
prнту	B

A user-defined function is called much as in C: first the arguments are evaluated; their values are the *actual parameters*. Then the function's body is evaluated with each actual parameter "bound to" (which is to say, named by) the corresponding *formal parameter* from the *formals* in the function's definition. In the first example, the actual parameter is 4, and $(+ n 1)$ is evaluated with 4 bound to n . In the second, the actual parameter is 7, and $(+ n n)$ is evaluated with 7 bound to n .

In Impcore, a function call behaves nicely only if the number of actual parameters is exactly equal to the number of formal parameters in the function's definition. Otherwise, the call goes wrong:

```
22a. <transcript 12a> +≡                                     <21g 22b>
      -> (add1 17 12)
      Run-time error: in (add1 17 12), expected 1 argument but found 2
```

In our examples so far, as in C, the name n means different things in different contexts. For example, functions `add1` and `double` use the name n as a formal parameter, and inside each function, n refers to that parameter. But outside any function, n continues to refer to the global variable defined on page 12. The meaning of any occurrence of any name is determined by the context in which the name occurs: If a variable x occurs in a function definition, and if the function has a formal parameter named with the same x , then x refers to that formal parameter. Otherwise x refers to a global variable. (Impcore has no local variables; to provide them is the object of Exercise 30.) If x occurs in a top-level expression, outside of any function definition, it necessarily refers to a global variable.

Let's contrive an example:

```
22b. <transcript 12a> +≡                                     <22a 23a>
      -> n                                                    ;; the global n
      7
      -> (define addn (n m) (set n (+ n m))) ;; mutates the parameter
      addn
      -> (addn n 1)                                           ;; the parameter is set to 8
      8
      -> n                                                    ;; the global n is unchanged
      7
```

Within the body of `addn`, the two occurrences of n refer to the formal parameter. But in the top-level expressions n and $(\text{addn } n \ 1)$, n refers to the global variable. And in the body of `addn`, where n is set, changing the formal n does not affect anything in the calling context; we say that Impcore passes parameters *by value*. No assignment to a formal parameter ever changes the value of a global variable.

With the details of expressions explored, we turn our attention to definitions. When a definition is evaluated, no value is returned; instead, evaluating a definition updates some part of the interpreter's state, causing it to "remember" something. The Impcore interpreter can remember global variables, function definitions, and pending unit tests.

- To evaluate a definition of the form $(\text{val } x \ e)$, we first check to see if a global variable named x exists, and if not, we create one. We then evaluate e and assign its value to x .
- To evaluate a definition that takes the form of an expression e , we evaluate e and store the result in the global variable `it`.
- To evaluate a definition of the form $(\text{define } f \ (x_1 \cdots x_n) \ e)$, like the definitions of `add1` or `double`, we remember f as a function that takes arguments $x_1, \dots, x_n$ and returns e .

- To evaluate a definition of the form (use *filename*), we look for a file called *filename*, which should contain a sequence of Impcore definitions. We read the definitions and evaluate them in order. And after reading the file, we run any unit tests it contains.
- To evaluate a definition of the form (check-expect e_1 e_2), we remember this test: at the end of the file containing the definition, evaluate both e_1 and e_2 . If their values are equal, the test passes; if not, the test fails.
- To evaluate a definition of the form (check-assert e), we remember this test: at the end of the file containing the definition, evaluate e . If its value is nonzero, the test passes; if not, the test fails.
- To evaluate a definition of the form (check-error e), we remember this test: at the end of the file containing the definition, evaluate e . If evaluating e triggers a run-time error, like dividing by zero or passing the wrong number of arguments to a function, the test passes; if not, the test fails.

1.2.4 What the primitive functions do

Not every function is defined using `define`; some are built into the interpreter as *primitives*. Each primitive function takes two arguments, except the printing primitives, which take one each. The arithmetic primitives `+`, `-`, `*`, and `/` do arithmetic on integers, up to the limits imposed by a 32-bit representation. Each of the comparison primitives `<`, `>`, and `=` does a comparison: if the comparison is true, the primitive returns 1; otherwise, it returns 0.

The printing primitives demand detailed explanation. Primitive `println` prints a value and then a newline; it's the printing primitive you'll use most often. Primitive `print` prints a value and *no* newline. Primitive `printu` prints a Unicode character and no newline. More precisely, `printu` takes as its argument an integer that stands for a Unicode *code point*—that means it's an integer code that stands for a character in one of a huge variety of alphabets. Primitive `printu` then prints the UTF-8 byte sequence that represents the code point. In most programming environments, this sequence will give you the character you're looking for. For example, `(printu 955)` prints the Greek letter λ . Each printing primitive, in addition to its side effect, also returns its argument.

When used interactively, the printing primitives can be confusing, because whenever an expression is evaluated, its value is printed automatically by the interpreter. Don't be baffled by effects like these:

```
23a. <transcript 12a>+≡                                     <122b 23b>
-> (val x 4)
-> (println x)
4
4
```

The 4 is printed twice because `println` is called with actual parameter 4 (the value of `x`), and `println` first *prints* 4, accounting for the first 4, then *returns* 4. The second 4 is printed because the interpreter prints the value of *every* expression, including `(println x)`.

```
23b. <transcript 12a>+≡                                     <123a 24a>
-> (val y 5)
5
-> (begin (println x) (println y) (* x y))
4
5
20
```

True definitions and extended definitions: A design compromise

If you want to learn to use programming languages well and also to describe them precisely, what languages should you study? Not big industrial languages—you can write interesting programs, but it’s hard to say how programs behave, or even what programs are well behaved. And not a tiny artificial language (or a “core calculus” like the famous *lambda calculus*)—its behavior can be described very precisely, but it’s hard to write any interesting programs. That’s why I’ve designed the bridge languages: to bridge the gap between industrial languages and core calculi.

The bridge languages are small enough to be described precisely, but big enough for interesting programs. (I won’t pretend you can write interesting programs in Impcore, but you *can* write interesting programs in μ Scheme, μ ML, Molecule, and μ Smalltalk.) But a few features are too complicated to define precisely, yet too useful to leave out. They are the “extended definitions”: the use, check-expect, check-assert, and check-error forms. The val, define, and top-level expression forms, which *are* defined precisely, are the “true definitions.” The true definitions are part of the language, and the extensions are there to make you more productive as a programmer.

If you happen to use print instead of println, you can get some strange output:

```
24a. <transcript 12a> +=                                     <23b 25b>
      -> (begin (print x) (print y) (* x y))
      4520
```

Because the interpreter automatically prints the value of each expression you enter, you’ll use printing primitives rarely—mostly for debugging. I typically debug using println, but when I want fancier output, I also use printu and print.

1.2.5 Extended definitions: Beyond interactive computation

The examples above show transcripts of my interactions with the Impcore interpreter. But interactive code disappears as soon as it is typed; to help you write code that you want to edit or keep, the Impcore interpreter, like all the interpreters in this book, enables you to put it in a file. And when you put code in a file, you can add *unit tests*. For example, the file gcd.imp tests a function that computes greatest common denominators:

```
24b. <contents of file gcd.imp 24b> +=                               25a>
      (val r 0)
      (define gcd (m n)
        (begin
          (while (!= (set r (mod m n)) 0)
            (begin
              (set m n)
              (set n r)))
          n))

      (check-expect (gcd 6 15) 3)
```

Since the code is in a file, there are no arrow prompts. But there is a unit test, our first: the check-expect says that if we call (gcd 6 15), the result should be 3.

The file includes more unit tests:

```
25a. <contents of file gcd.imp 24b>+≡                                     <24b>
  (check-expect (gcd 15 15) 15)
  (check-expect (gcd 14 15) 1)
  (check-expect (gcd 14 1) 1)
  (check-expect (gcd 72 96) 24)
  (check-error (gcd 14 0))
```

The last unit test says that if we evaluate (gcd 14 0), we expect a run-time error.

Unit tests aren't run until after a file is loaded. Then the interpreter summarizes the test results:

```
25b. <transcript 12a>+≡                                     <24a 25d>
  -> (use gcd.imp)
  0
  gcd
  All 6 tests passed.
```

A unit test can appear *before* the function it tests. This trick can be a great way to plan a function, or to document it. I've written an example using *triangular numbers*. A triangular number is analogous to the square of a number: just as the square of n is the number of dots needed to form a square array with a side of length n , the n th triangular number is the number of dots needed to form an equilateral triangle with n dots along one side.

```
1   =   *

3   =   *
      * *

      *

6   =   * *
      * * *
      * * *
```

A triangular number can be computed using the sigma function in Exercise 2 on page 74, but there's a shortcut:

```
25c. <triangle.imp 25c>≡
  (check-expect (triangle 1) 1)
  (check-expect (triangle 2) 3)
  (check-expect (triangle 3) 6)
  (check-expect (triangle 4) 10)

  (define triangle (n)
    (/ (* n (+ n 1)) 2))

25d. <transcript 12a>+≡                                     <25b 26a>
  -> (use triangle.imp)
  triangle
  All 4 tests passed.
```

When writing triangle, I botched my first attempt. My unit tests caught the botch. The botched code, preceded by tests, looked like this:

```
25e. <botched-triangle.imp 25e>≡
  (check-expect (triangle 1) 1)
  (check-expect (triangle 2) 3)
  (check-expect (triangle 3) 6)
  (check-expect (triangle 4) 10)

  (define triangle (n)      ; botched version
    (/ (* n (- n 1)) 2))
```

The unit tests caught the botch:

```
26a. <transcript 12a>+≡                                     <125d 26c>
-> (use botched-triangle.imp)
triangle
Check-expect failed: expected (triangle 1) to evaluate to 1, but it's 0.
Check-expect failed: expected (triangle 2) to evaluate to 3, but it's 1.
Check-expect failed: expected (triangle 3) to evaluate to 6, but it's 3.
Check-expect failed: expected (triangle 4) to evaluate to 10, but it's 6.
All 4 tests failed.
```

The check-expect form provides what you'll need to test most Impcore code: because every Impcore value is an integer, your unit tests can usually just say what value you expect. But in Chapter 2 and beyond, you'll have access to more structured values, like records and lists, and you'll want to talk about *properties*, like “the list is sorted.” For that purpose, each of the bridge languages includes a check-assert form, which is intended for use with Booleans. As examples, here are a few assertions about properties of products.

```
26b. <arith-assertions.imp 26b>≡
  (check-assert (< (* 1 2) (+ 1 2))) ; product is smaller than sum
  (check-assert (> (* 2 3) (+ 2 3))) ; product is bigger than sum
  (check-assert (not (< (* -1 -1) 0))) ; product of negatives is not negative
```

Function not is predefined; its definition appears in Figure 1.3 (page 27).

```
26c. <transcript 12a>+≡                                     <126a 29>
-> (use arith-assertions.imp)
All 3 tests passed.
```

1.2.6 Primitive, predefined, and basis; the initial basis

Programmers like big languages with lots of data types and syntactic forms, but implementors want to keep primitive functionality small and simple. (So do semanticists!) To reconcile these competing desires, language designers have found two strategies: translation into a *core language* and definition of an *initial basis*.

Using a core language, you stratify your language into two layers. The inner layer defines or implements its constructs directly; it constitutes the core language. The outer layer defines additional constructs by translating them into the core language; these constructs constitute *syntactic sugar*. In this chapter, the core language is Impcore, and as an example of syntactic sugar, a for expression can be defined by a translation into begin and while (Section 1.8).

To be useful to programmers, a language needs to be accompanied by a standard library. In the theory world, a library contributes to a *basis*; basis is the collective term for all the things that can be named in definitions. In Impcore, these things are functions and global variables. A language's *initial basis* contains the named things that are available in a fresh interpreter or installation—the things you have access to even before evaluating your own code.

Like the Impcore language, Impcore's initial basis is stratified into two layers, one of which is defined in terms of the other. The inner layer includes all the functions that are defined directly by C code in the interpreter; these are called *primitive*. The outer layer includes functions that are also built into the interpreter, but are defined in terms of the primitives using Impcore source code; they are user-defined functions and are called *predefined*.

Stratifying the initial basis makes life easy for everyone. Implementors make their own lives easy by defining just a few primitives, and they can make programmers' lives easy by defining lots of predefined functions. Predefined functions are

Boolean connectives are defined using `if` expressions.

27a. *(predefined Impcore functions 27a)* $\equiv$

27b▷

```
(define and (b c) (if b c b))
(define or  (b c) (if b b c))
(define not (b)  (if b 0 1))
```

Unlike the similar constructs built into the syntax of many languages, these versions of `and` and `or` always evaluate both of their arguments. Section G.7 shows how you can use syntactic sugar to define *short-circuit* variations that evaluate a second expression only when necessary.

Only comparisons `<`, `=`, and `>` are primitive; the others are predefined.

27b. *(predefined Impcore functions 27a)* $+\equiv$

◁27a 27c▷

```
(define <= (x y) (not (> x y)))
(define >= (x y) (not (< x y)))
(define != (x y) (not (= x y)))
```

Primitive arithmetic includes `+`, `-`, `*`, and `/`, to which the predefined functions add modulus and negation.

27c. *(predefined Impcore functions 27a)* $+\equiv$

◁27b

```
(define mod (m n) (- m (* n (/ m n))))
(define negated (n) (- 0 n))
```

These functions are installed into the initial basis by the C code in chunk *(install the initial basis in functions S297b)*, which is continued in chunk S297d.

Figure 1.3: Predefined functions in Impcore's initial basis

just ordinary code, and writing them is lots easier than defining new primitives. Impcore's predefined functions are defined by the code in Figure 1.3.

Just like any other function, a primitive or predefined function can be redefined using `define`. This trick can be useful—for example, to count the number of times a function is called. But if you redefine an initial-basis function, don't change the results it returns! (You'll introduce bugs.)

Before going on to the next sections, work some of the exercises in Sections 1.10.2 to 1.10.4, starting on page 73.

1.3 ABSTRACT SYNTAX

In every interpreter in this book, programs are represented as *abstract-syntax trees* (ASTs). And as noted in Section 1.1, abstract syntax isn't just a great representation; abstract-syntax trees are the best way to *think* about syntax. Alas, in C, AST representations are a pain to code. To ease the pain, the representations in this book are generated from a little language that is used only to describe abstract-syntax trees. This language also helps with our thinking. As an example, the language describes the abstract syntax of an Impcore definition like this:

27d. *(simplified example of abstract syntax for Impcore 27d)* $\equiv$

28▷

```
Def = VAL      (Name, Exp)
    | EXP      (Exp)
    | DEFINE   (Name, Namelist, Exp)
```

This description defines `Def` as a category of abstract-syntax tree, which includes only the true definitions; extended definitions are `XDefs`. Each form of `Def` is named (`VAL`, `EXP`, or `DEFINE`), and following the name of the form is a comma-separated list showing what values or subtrees each form is made from.

The same language is used to specify the abstract representation of an expression, which has more forms:

28. $\langle \text{simplified example of abstract syntax for Impcore 27d} \rangle + \equiv$

$\triangleleft 27d$

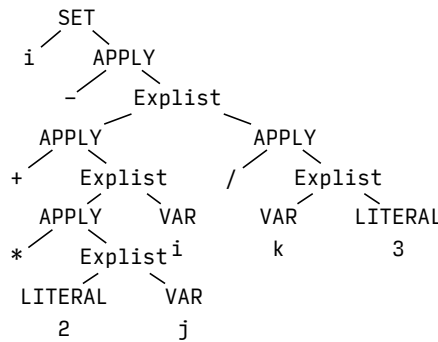
```
Exp = LITERAL (Value)
      | VAR      (Name)
      | SET      (Name, Exp)
      | IF       (Exp, Exp, Exp)
      | WHILE    (Exp, Exp)
      | BEGIN    (Explist)
      | APPLY    (Name, Explist)
```

An imperative core

28

This description is worth comparing this with the description of *exp* in the grammar for Impcore (Figure 1.1, page 18).

An abstract-syntax tree for a given form is usually drawn with the form's name at the root and with its subtrees connected to it by solid lines. When a form has no subtrees, like LITERAL or VAR, its single node can be drawn with the associated Value or Name just underneath. For example, the abstract-syntax tree for the Impcore expression (set i (- (+ (* 2 j) i) (/ k 3))) can be drawn like this:



An abstract-syntax tree is much easier to analyze, manipulate, or interpret than source code. And ASTs focus our attention on structure and semantics. Concrete syntax becomes a separate concern; one could easily define a version of Impcore with C-like concrete syntax, but with identical abstract syntax.

Abstract syntax is created from concrete input by a *parser*, which also identifies and rejects ill-formed phrases such as (if x 0) or (val y). Parsing is covered in a large body of literature; for textbook treatments, try Appel (1998) or Aho et al. (2007). A parser for Impcore, which you can easily extend, can be found in Appendix G.

1.4 ENVIRONMENTS AND THE MEANINGS OF NAMES

An expression like (* x 3) cannot be evaluated by itself; to know its value, we must know what x is. Similarly, to know the effect of evaluating an expression like (set x 1), we must know whether x is a formal parameter, a global variable, or something else entirely. In programming-language theory, knowledge about names is kept in an *environment*, which is usually a mapping from names to meanings. Environments are often implemented as hash tables or search trees, and in an implementation, an environment is sometimes called a *symbol table*.

Knowing what can be in an environment tells us what kinds of things a name can stand for, which is a good first step in understanding a new programming language. To understand Impcore, we need to know about three environments, each of which is written using its own metavariable:

- Environment ξ (x_i , pronounced “ksee”) holds values of global variables.

- Environment ϕ (phi, pronounced “fee”) holds definitions of functions.
- Environment ρ (rho, pronounced “roe”) holds values of a function’s parameters.

Environments ξ and ϕ are global and shared, but there is a distinct ρ for every function call. Together, the contents of the three environments comprise Impcore’s basis.

A name can be defined in all three environments at once. But it’s a bad idea:

29. $\langle \text{transcript 12a} \rangle + \equiv$ $\triangleleft 26c \ 64 \triangleright$
 -> (val x 2)
 2
 -> (define x (y) (+ x y)) ; pushing the boundaries of knowledge...
 -> (define z (x) (x x)) ; and sanity
 -> (z 4)
 6

The val definition introduces a global variable x, which is bound to value 2 in environment ξ . The first define introduces a function x that adds its argument to the global variable x; that function is bound to name x in environment ϕ . The second define introduces function z, which passes its formal parameter x to the function x; when z is called, parameter x is bound to 4 in environment ρ . This example should push you to understand the rules of Impcore; to follow it, you have to know not only that Impcore has three environments but also how the environments are used. Of course, no sane person programs this way; production code is written to be *easy* to understand, even by readers who may have forgotten details of the rules.

Mathematically, an environment is a function with a finite domain, from a name to whatever. In Impcore, environments ξ and ρ map each defined name to a value; ϕ maps each defined name to a function. Whatever a name is mapped to, all environments are manipulated using the same notation, which is mostly function notation. For example, whatever is associated with name x in the environment ρ is written $\rho(x)$. The set of names bound in environment ρ is written $\text{dom } \rho$. An extended environment, ρ plus a binding of the name x to v , is written $\rho\{x \mapsto v\}$. In an extended environment, the new binding hides previous bindings of x , so lookup is governed by this equation:

$$\rho\{x \mapsto v\}(y) = \begin{cases} v, & \text{when } y = x \\ \rho(y), & \text{when } y \neq x. \end{cases}$$

Finally, an empty environment, which does not bind any names, is written $\{\}$.

One environment can be combined with another, but because combining environments is not useful in Impcore, the notation is deferred to Chapter 2.

1.5 OPERATIONAL SEMANTICS

Earlier in this chapter, Impcore is defined informally and concisely, if imprecisely (Section 1.2). For a precise understanding of what Impcore is, we could consult the implementation (Section 1.6), but an implementation is much longer and harder to understand than an informal description. An implementation also lacks focus; code embodies many irrelevant decisions, including decisions about the *representations* of names, environments, and abstract syntax. Those decisions are part of the implementation, not part of the language.

In this section, Impcore is defined using a technique that is concise, focused, and precise: formal *operational semantics*. Operational semantics specifies behavior precisely while hiding implementation details. Although the notation can be

Table 1.4: Metavariables of Impcore’s operational semantics

e, e_i	An expression
d	A definition
x, x_i	A name that refers to a variable or a parameter
f	A name that refers to a function
v, v_i	A value
$\xi, \xi', \dots$	A global-variable environment
$\phi, \phi', \dots$	A function-definition environment
$\rho, \rho', \dots$	A formal-parameter environment

1

intimidating at first, with practice operational semantics becomes as easy to read as informal English.

Impcore’s operational semantics defines an *abstract machine* and rules for its execution. The semantics defines the machine’s states, including its start state and its acceptable final states, and it presents rules for making transitions from one state to another. By applying these rules repeatedly, the machine can go from a start state like “I just turned on and have this program to evaluate” to an accepting state like “the answer is 42.”

A machine may reach a state from which it cannot make progress; for example, a machine evaluating $(/ \ 1 \ 0)$ probably cannot make a transition. When a machine reaches such a state, we say it “gets stuck” or “goes wrong.” An implementation might indicate a run-time error.

An Impcore machine’s states and transitions are described using the metavariables from Sections 1.2 and 1.4, which describe syntax, values, and semantics (Table 1.4). The state of an Impcore machine has four parts: a *definition* d or *expression* e being evaluated; a value environment ξ , which holds the values of global variables; a function-definition environment ϕ ; and a value environment ρ , which holds the values of formal parameters. Definitions do not appear inside functions, so when the machine is evaluating a definition d , there are no formal parameters, and its state is written as $\langle d, \xi, \phi \rangle$. When the machine is evaluating an expression e , its state is written as $\langle e, \xi, \phi, \rho \rangle$. When the machine is resting between evaluations, it remembers only the values of global variables and the definitions of functions, and its state is written as $\langle \xi, \phi \rangle$.

Environments ξ and ρ store values, which in Impcore are integers. Environment ϕ stores both primitive functions and user-defined functions. A primitive function is written as $\text{PRIMITIVE}(\oplus)$, where $\oplus$ stands for a name like $+$, $=$, or $*$. A user-defined function is written as $\text{USER}(\langle x_1, \dots, x_n \rangle, e)$, where the x_i ’s are the formal parameters and e is the body.

In the initial state of the Impcore machine, $\xi = \{\}$, because there are no global variables defined. But $\phi = \phi_0$, where ϕ_0 is preloaded with the definitions of primitive functions as well as the user-defined functions in the initial basis (Figure 1.3 on page 27).

1.5.1 Judgments and rules of inference

State transitions are described using *judgments*, which take one form for definitions and another form for expressions. The judgment for the evaluation of an expression, $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle$, means “evaluating expression e produces value v .” More precisely, it means “in environments ξ , ϕ , and ρ , evaluating e produces a value v , and it also produces new environments ξ' and ρ' , while leaving ϕ

unchanged.”² This judgment uses eight metavariables; e stands for an expression, ξ , ϕ , and ρ stand for environments, and v stands for a value. Different metavariables are distinguished by giving them subscripts, or as with the environments, by using primes.

The form of the judgment $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle$ tells us a few things:

- Evaluating an expression always produces a value, unless of course the machine gets stuck. Even expressions like SET and WHILE, which are typically evaluated only for side effects, produce values.³
- Evaluating an expression might change the value of a global variable (from ξ) or a formal parameter (from ρ).
- Evaluating an expression never adds or changes a function definition (because ϕ is unchanged).

The form of the judgment *doesn't* tell us whether evaluating an expression can introduce a new variable. (It can't, but we can be certain of this only if we study the full semantics and write an inductive proof, which is Exercise 24 on page 83.)

The form of the evaluation judgment also gives this semantics part of its name: no matter how much computation is required to get from e to v , the judgment $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle$ encompasses all that computation in one big step. It is therefore called a *big-step* judgment and is part of a *big-step* semantics.

The judgment for a definition is simpler; $\langle d, \xi, \phi \rangle \rightarrow \langle \xi', \phi' \rangle$ means “evaluating definition d in the environments ξ and ϕ yields new environments ξ' and ϕ' .” The different arrow helps distinguish this judgment from an expression judgment.

Not all judgments describe real program behaviors. For example, unless some joker changes the binding of the name $+$ in ϕ , $\langle (+\ 1\ 1), \xi, \phi, \rho \rangle \Downarrow \langle 4, \xi, \phi, \rho \rangle$ doesn't describe how Impcore code behaves. To say which judgments describe real behaviors, an operational semantics uses *rules of inference*. Each rule has the form

$$\frac{\text{premises}}{\text{conclusion}}. \quad (\text{NAME OF RULE})$$

If all the premises hold, so does the conclusion.

For example, the rule

$$\frac{\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v_1, \xi', \phi, \rho' \rangle \quad v_1 \neq 0 \quad \langle e_2, \xi', \phi, \rho' \rangle \Downarrow \langle v_2, \xi'', \phi, \rho'' \rangle}{\langle \text{IF}(e_1, e_2, e_3), \xi, \phi, \rho \rangle \Downarrow \langle v_2, \xi'', \phi, \rho'' \rangle}, \quad (\text{IFTRUE})$$

which is part of the semantics of Impcore, says that whenever $\langle e_1, \xi, \phi, \rho \rangle$ evaluates to some nonzero value v_1 , the expression $\text{IF}(e_1, e_2, e_3)$ evaluates to the result of evaluating e_2 . Because e_2 is evaluated in the environment produced by evaluation of e_1 , if e_1 contains side effects, such as assigning to a variable, the results of those side effects are visible to e_2 . Expression e_3 is not mentioned, because when $v_1 \neq 0$, e_3 is never evaluated.

²A judgment describes a relation, not a function. In principle, it is possible to have two different values $v_1 \neq v_2$ such that $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v_1, \xi', \phi, \rho' \rangle$ and also $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v_2, \xi', \phi, \rho' \rangle$. A language that permits such ambiguity is *nondeterministic*. All the languages in this book are deterministic, but multithreaded languages like Java or C# can be nondeterministic. Programs written in such languages can produce different answers on different runs. Languages that do not specify the order in which expressions are evaluated, like C, can also be nondeterministic. Programs written in such languages can produce different answers when translated with different compilers. (See also Exercise 27, page 84.)

³This property distinguishes an expression-oriented language like Impcore (or ML, Scheme, or Smalltalk) from a statement-oriented language like C. Both have imperative constructs that are evaluated only for side effects, but only in C do these constructs produce no values.

The rule specifies that e_1 is evaluated before e_2 . How do we know? By looking at which environments go where. The side effects of e_1 are captured in environments ξ' and ρ' , and these are the environments used to evaluate e_2 . Order of evaluation is determined not by the order in which the premises are written, but by the flow of data (in this case, the environments) through the computation. That said, the rule's premises are written in the same order as the evaluations they describe. Any other order would be, shall we say, discourteous.

The rules of Impcore's semantics belong to a larger family of reasoning techniques called *natural deduction*. This family gives Impcore's semantics the rest of its name: it is a *big-step*, *natural-deduction* semantics.

Rules of inference can be translated into code. A complete example is done for you in Section 1.6: Impcore's expression-evaluation judgment is implemented by function `eval`. Calling `eval(e,  $\xi$ ,  $\phi$ ,  $\rho$ )` returns v and has side effects on ξ and ρ with the result that $\langle e, \xi_{\text{before}}, \phi, \rho_{\text{before}} \rangle \Downarrow \langle v, \xi_{\text{after}}, \phi, \rho_{\text{after}} \rangle$. Function `eval` looks at the form of e and considers rules that have e 's of that form in their conclusions. It then looks at premises; each evaluation judgment in a premise is implemented by a recursive call. For example, to evaluate an IF expression, `eval` first makes a recursive call to itself to find v_1 , ξ' , and ρ' such that $\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v_1, \xi', \phi, \rho' \rangle$. Then if $v_1 \neq 0$, `eval` makes another recursive call to find v_2 , ξ'' , and ρ'' such that $\langle e_2, \xi', \phi, \rho' \rangle \Downarrow \langle v_2, \xi'', \phi, \rho'' \rangle$. Having satisfied all the premises of rule IFTRUE, it then returns v_2 and the modified environments.

The rules for every possible form of expression (and definition) are presented in the rest of this section.

1.5.2 Literal values

A literal value evaluates to itself, without changing any environments:

$$\frac{}{\langle \text{LITERAL}(v), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi, \phi, \rho \rangle} \quad (\text{LITERAL})$$

This rule has no premises, so at a LITERAL node, the recursive implementation of `eval` terminates.

1.5.3 Variables

If a name is bound in the parameter or global environment, then evaluating the variable with that name produces the value associated with it in the environment. Otherwise, no rules apply, the machine gets stuck, and the computation does not continue.

Parameters hide global variables. Name x is a parameter if $x \in \text{dom } \rho$, where $\text{dom } \rho$ is the domain of ρ , i.e., the set of names bound by ρ .

$$\frac{x \in \text{dom } \rho}{\langle \text{VAR}(x), \xi, \phi, \rho \rangle \Downarrow \langle \rho(x), \xi, \phi, \rho \rangle} \quad (\text{FORMALVAR})$$

$$\frac{x \notin \text{dom } \rho \quad x \in \text{dom } \xi}{\langle \text{VAR}(x), \xi, \phi, \rho \rangle \Downarrow \langle \xi(x), \xi, \phi, \rho \rangle} \quad (\text{GLOBALVAR})$$

These rules have premises that involve only membership tests on environments, not other evaluation judgments, so at a VAR node, the recursive implementation of `eval` terminates.

Notation can imply “may differ” or “must equal” but not “must differ”

When we specify evaluation using a judgment form, we are using a form of mathematical logic, which uses primes and subscripts in ways that can be hard to learn. When primes or subscripts differ, the metavariables they decorate *may* or *may not* differ, like the ξ and ξ' in these examples:

- *The general form of judgment* $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle$
The general form uses ξ on the left and ξ' on the right. Environments ξ and ξ' look different, but the notation means only that they *may differ*. The form says, “evaluating an unknown expression e *may* change a global variable.” What actually happens depends on what rules you use and what judgment you prove.
- *Conclusion of LITERAL* $\langle \text{LITERAL}(v), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi, \phi, \rho \rangle$
The LITERAL rule proves a judgment that uses ξ on both the left and the right. Because ξ *must equal* ξ , the judgment says, “evaluating a literal expression $\text{LITERAL}(v)$ *does not* change or add a global variable.”
- *Conclusion of GLOBALASSIGN* $\langle \text{SET}(x, e), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi'\{x \mapsto v\}, \phi, \rho' \rangle$
Environments ξ and $\xi'\{x \mapsto v\}$ look like they *must differ*. But it’s not so: ξ may equal ξ' , and ξ' may map x to v . In that case, ξ and $\xi'\{x \mapsto v\}$ are equal. Imagine evaluating $(\text{set } x \ 0)$ when x is *already* zero.

Our notation can imply only that two environments “may differ” or “must equal” each other. For example, in the rule for IFTRUE on page 34, ξ may equal ξ' , which may equal ξ'' , as in $(\text{if } (> n \ 0) \ n \ (- \ 0 \ n))$. Or ξ may equal ξ' , but they may both differ from ξ'' , as in $(\text{if } (> n \ 0) \ (\text{set sign } 1) \ (\text{set sign } -1))$.

What if ξ and ξ' *must differ*? That can’t be said with primes or subscripts; primes and subscripts can say only “may differ” or “must equal.” To say that two things *must differ*, write an explicit premise, like “ $v_1 \neq 0$ ” or “ $\xi \neq \xi'$.”

The notation is most important when you write your own derivations. Beginners often write primes or subscripts as they appear in rules. But a prime or subscript says “I don’t know; it may differ,” and in a derivation, this is almost always wrong—we *do* know. Here’s an invalid derivation, intended to describe the evaluation of $(\text{if } (> n \ 0) \ n \ (- \ 0 \ n))$ when n is 7 (I’ve taken a minor liberty with the notation):

$$\frac{\begin{array}{c} \dots \\ \overline{\langle (> n \ 0), \xi, \phi, \rho \rangle \Downarrow \langle 1, \xi, \phi, \rho \rangle} \end{array} \quad 1 \neq 0 \quad \begin{array}{c} \dots \\ \overline{\langle n, \xi, \phi, \rho \rangle \Downarrow \langle 7, \xi, \phi, \rho \rangle} \end{array}}{\langle \text{IF}(> n \ 0), n, (- \ 0 \ n), \xi, \phi, \rho \rangle \Downarrow \langle 7, \xi'', \phi, \rho'' \rangle}.$$

The conclusion is bogus. Because ξ'' appears only in the conclusion, there is nothing that ξ'' must equal, and that means the judgment says, “evaluating the *if* expression produces 7, and afterward, global variables have *arbitrary* values.” (The same goes for ρ'' .) To avoid this problem, remember that in any judgment that you prove, the elements of the final state *must equal* something that you specify. Do that and you’ll write good derivations.

1.5.4 Assignment

Evaluating an assignment changes the value of a variable, and it produces the value of the right-hand side. Assignment, like variable lookup, prioritizes parameters: if the name x is bound in the parameter environment ρ , its binding is changed there; otherwise, x had better be bound in the global environment ξ .

$$\frac{x \in \text{dom } \rho \quad \langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}{\langle \text{SET}(x, e), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \{x \mapsto v\} \rangle} \quad (\text{FORMALASSIGN})$$

$$\frac{x \notin \text{dom } \rho \quad x \in \text{dom } \xi \quad \langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}{\langle \text{SET}(x, e), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi' \{x \mapsto v\}, \phi, \rho' \rangle} \quad (\text{GLOBALASSIGN})$$

Each of these rules has a premise that shows an evaluation of the right-hand side e , so at a SET node, the recursive implementation of eval always makes a recursive call. And even though there are two rules with SET in the conclusion, only one can apply at one time, because the premises $x \in \text{dom } \rho$ and $x \notin \text{dom } \rho$ are mutually exclusive. This property enables the implementation to know exactly what to do with a SET node, and it keeps the evaluation of Impcore programs deterministic.

Because of the premises $x \in \text{dom } \rho$ and $x \in \text{dom } \xi$, only a previously defined variable may be assigned to; given a SET node where $x \notin \text{dom } \rho$ and $x \notin \text{dom } \xi$, the machine gets stuck. In many languages, like Awk for example, assignment to an undefined and undeclared variable creates a new global variable (Aho, Kernighan, and Weinberger 1988), as specified by the following rule:

$$\frac{x \notin \text{dom } \rho \quad \langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}{\langle \text{SET}(x, e), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi' \{x \mapsto v\}, \phi, \rho' \rangle} \quad (\text{GLOBALASSIGN for Awk})$$

In Impcore, a new global variable can be created only by a VAL definition, as shown below in rule DEFINEGLOBAL (page 37). To spot such subtleties, you have to read inference rules carefully.

1.5.5 Control flow

Conditional evaluation

The expression IF(e_1, e_2, e_3) first evaluates the expression e_1 to produce value v_1 . If v_1 is nonzero, the IF expression produces the result of evaluating e_2 ; otherwise, it produces the result of evaluating e_3 . Each case is described by its own rule.

$$\frac{\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v_1, \xi', \phi, \rho' \rangle \quad v_1 \neq 0 \quad \langle e_2, \xi', \phi, \rho' \rangle \Downarrow \langle v_2, \xi'', \phi, \rho'' \rangle}{\langle \text{IF}(e_1, e_2, e_3), \xi, \phi, \rho \rangle \Downarrow \langle v_2, \xi'', \phi, \rho'' \rangle} \quad (\text{IFTRUE})$$

$$\frac{\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v_1, \xi', \phi, \rho' \rangle \quad v_1 = 0 \quad \langle e_3, \xi', \phi, \rho' \rangle \Downarrow \langle v_3, \xi'', \phi, \rho'' \rangle}{\langle \text{IF}(e_1, e_2, e_3), \xi, \phi, \rho \rangle \Downarrow \langle v_3, \xi'', \phi, \rho'' \rangle} \quad (\text{IFFALSE})$$

Like the two rules for SET, these rules have mutually exclusive premises ($v_1 \neq 0$ and $v_1 = 0$). Evaluation is deterministic, and by examining v_1 , the implementation knows which rule to apply.

A WHILE loop first evaluates the condition e_1 to produce value v_1 . If v_1 is nonzero, evaluation continues with the body e_2 , and then the WHILE loop is evaluated again.

$$\frac{\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v_1, \xi', \phi, \rho' \rangle \quad v_1 \neq 0 \quad \langle e_2, \xi', \phi, \rho' \rangle \Downarrow \langle v_2, \xi'', \phi, \rho'' \rangle \quad \langle \text{WHILE}(e_1, e_2), \xi'', \phi, \rho'' \rangle \Downarrow \langle v_3, \xi''', \phi, \rho''' \rangle}{\langle \text{WHILE}(e_1, e_2), \xi, \phi, \rho \rangle \Downarrow \langle v_3, \xi''', \phi, \rho''' \rangle} \quad (\text{WHILEITERATE})$$

In this rule, the value v_2 , which is produced by evaluating the body e_2 , is thrown away. We can tell by looking at the final state of the judgment in the conclusion of the rule. That state has elements v_3 , ξ''' , ϕ , and ρ''' , and if you study the rule carefully, you'll see that none of these elements depends on or uses v_2 . A WHILE loop evaluates its body e_2 only for its side effects, i.e., for the new environments ξ'' and ρ'' . They are then used to make the final environments ξ''' and ρ''' .

If the condition in a WHILE loop evaluates to zero, the loop terminates, and the loop also evaluates to zero.

$$\frac{\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v_1, \xi', \phi, \rho' \rangle \quad v_1 = 0}{\langle \text{WHILE}(e_1, e_2), \xi, \phi, \rho \rangle \Downarrow \langle 0, \xi', \phi, \rho' \rangle} \quad (\text{WHILEEND})$$

If the evaluation of a WHILE loop terminates, it always produces zero, even when rule WHILEITERATE is used (Exercise 23 on page 83). A WHILE loop is therefore executed for its side effects.

Sequential execution

A nonempty BEGIN evaluates its subexpressions left to right, producing the result of the final expression e_n .

$$\frac{\begin{array}{c} \langle e_1, \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v_1, \xi_1, \phi, \rho_1 \rangle \\ \langle e_2, \xi_1, \phi, \rho_1 \rangle \Downarrow \langle v_2, \xi_2, \phi, \rho_2 \rangle \\ \vdots \\ \langle e_n, \xi_{n-1}, \phi, \rho_{n-1} \rangle \Downarrow \langle v_n, \xi_n, \phi, \rho_n \rangle \end{array}}{\langle \text{BEGIN}(e_1, e_2, \dots, e_n), \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v_n, \xi_n, \phi, \rho_n \rangle} \quad (\text{BEGIN})$$

Values v_1 to v_{n-1} are ignored, but the environments ξ_1 and ρ_1 , which result from evaluating e_1 , are used to evaluate e_2 , and so on. The use of ξ_1 and ρ_1 to evaluate e_2 implies that e_1 is evaluated before e_2 . Order of evaluation is determined by this “threading” of environments, not by the order in which the premises are written. For example, the BEGIN rule might equally well have been written this way:

$$\frac{\begin{array}{c} \langle e_n, \xi_{n-1}, \phi, \rho_{n-1} \rangle \Downarrow \langle v_n, \xi_n, \phi, \rho_n \rangle \\ \langle e_{n-1}, \xi_{n-2}, \phi, \rho_{n-2} \rangle \Downarrow \langle v_{n-1}, \xi_{n-1}, \phi, \rho_{n-1} \rangle \\ \vdots \\ \langle e_1, \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v_1, \xi_1, \phi, \rho_1 \rangle \end{array}}{\langle \text{BEGIN}(e_1, e_2, \dots, e_n), \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v_n, \xi_n, \phi, \rho_n \rangle} \quad (\text{equivalent BEGIN})$$

This equivalent rule still specifies that e_1 is evaluated before e_2 , and so on, but when the rule is written this way, it is not as easy to understand.

A BEGIN might be empty, in which case it evaluates to zero.

$$\frac{}{\langle \text{BEGIN}(), \xi, \phi, \rho \rangle \Downarrow \langle 0, \xi, \phi, \rho \rangle} \quad (\text{EMPTYBEGIN})$$

1.5.6 Function application

In the rules for function application, operational semantics first starts to show its advantages: the description is precise, and it is much more concise than an implementation.

User-defined functions

Application of a user-defined function f evaluates argument expressions e_1 to e_n , then evaluates the body of f in a new environment that binds each formal parameter to the value of the corresponding argument.

$$\begin{array}{c}
 \phi(f) = \text{USER}(\langle x_1, \dots, x_n \rangle, e) \\
 x_1, \dots, x_n \text{ all distinct} \\
 \langle e_1, \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v_1, \xi_1, \phi, \rho_1 \rangle \\
 \vdots \\
 \langle e_n, \xi_{n-1}, \phi, \rho_{n-1} \rangle \Downarrow \langle v_n, \xi_n, \phi, \rho_n \rangle \\
 \frac{\langle e, \xi_n, \phi, \{x_1 \mapsto v_1, \dots, x_n \mapsto v_n\} \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}{\langle \text{APPLY}(f, e_1, \dots, e_n), \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v, \xi', \phi, \rho_n \rangle} \quad (\text{APPLYUSER})
 \end{array}$$

As in the BEGIN rule, expressions e_1 through e_n are evaluated in order. Their values v_1 to v_n are used to create a new, unnamed formal-parameter environment that maps each formal parameter x_i to the corresponding v_i . This environment is used as a ρ to evaluate e , the body of the function.

The rule has these implications:

- The behavior of a function doesn't depend on the function's name, but only on the definition to which the name is bound.
- The body of a function can't get at the formal parameters of its caller, since the body e is evaluated in a state that does not contain $\rho_0, \dots, \rho_n$.
- If a function assigns to its own formal parameters, its caller can't get at the new values, because the caller has no access to the environment ρ' .
- After the body of a function is evaluated, the environment ρ' containing the values of its formal parameters is thrown away. This fact matters to implementors of programming languages, who can use temporary space (in registers and on the stack) to implement formal-parameter environments.

In short, the formal parameters of a function are private to that function—neither its caller nor its callees can see them or modify them. The privacy of formal parameters is an essential part of what language designers call “functional abstraction,” which both programmers and implementors rely on.

Primitive functions

Application of a primitive function resembles the application of a user-defined function. The arguments are evaluated, and the application produces the result of performing a primitive operation on their values. In principle, each primitive is describe by its own rule or rules, but this book shows only a few representatives.

The arithmetic primitives are represented by addition:

$$\begin{array}{c}
 \phi(f) = \text{PRIMITIVE}(+) \\
 \langle e_1, \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v_1, \xi_1, \phi, \rho_1 \rangle \\
 \langle e_2, \xi_1, \phi, \rho_1 \rangle \Downarrow \langle v_2, \xi_2, \phi, \rho_2 \rangle \\
 - 2^{31} \leq v_1 + v_2 < 2^{31} \\
 \hline
 \langle \text{APPLY}(f, e_1, e_2), \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v_1 + v_2, \xi_2, \phi, \rho_2 \rangle
 \end{array} \quad (\text{APPLYADD})$$

The final condition on the sum $v_1 + v_2$ ensures that the result can be represented in a 32-bit signed integer.

Comparison primitives are represented by equality. Like the `if` expression, the equality primitive is specified by a pair of rules which have mutually exclusive premises ($v_1 = v_2$ and $v_1 \neq v_2$):

$$\begin{array}{c}
 \phi(f) = \text{PRIMITIVE}(=) \\
 \langle e_1, \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v_1, \xi_1, \phi, \rho_1 \rangle \\
 \langle e_2, \xi_1, \phi, \rho_1 \rangle \Downarrow \langle v_2, \xi_2, \phi, \rho_2 \rangle \\
 v_1 = v_2 \\
 \hline
 \langle \text{APPLY}(f, e_1, e_2), \xi_0, \phi, \rho_0 \rangle \Downarrow \langle 1, \xi_2, \phi, \rho_2 \rangle
 \end{array} \quad (\text{APPLYEQTRUE})$$

$$\begin{array}{c}
 \phi(f) = \text{PRIMITIVE}(=) \\
 \langle e_1, \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v_1, \xi_1, \phi, \rho_1 \rangle \\
 \langle e_2, \xi_1, \phi, \rho_1 \rangle \Downarrow \langle v_2, \xi_2, \phi, \rho_2 \rangle \\
 v_1 \neq v_2 \\
 \hline
 \langle \text{APPLY}(f, e_1, e_2), \xi_0, \phi, \rho_0 \rangle \Downarrow \langle 0, \xi_2, \phi, \rho_2 \rangle
 \end{array} \quad (\text{APPLYEQFALSE})$$

The printing primitives are represented by `println`. These primitives have an important behavior that can't be expressed by our formal evaluation judgment: they print. Because this behavior is omitted, the rule makes `println` look like the identity function.

$$\begin{array}{c}
 \phi(f) = \text{PRIMITIVE}(\text{println}) \\
 \langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle \\
 \hline
 \langle \text{APPLY}(f, e), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle \quad \text{while printing } v
 \end{array} \quad (\text{APPLYPRINTLN})$$

The printing primitives could be modeled formally without much trouble; for example, we could extend the abstract-machine state to include a sequence of all characters ever printed. I prefer, however, not to clutter our semantics with such a list. Leaving the specification of printing informal is OK because our semantics is intended to convey understanding, not to nail down every last detail.

1.5.7 Rules for evaluating definitions

The rules above specify the effects of evaluating expressions. The effects of evaluating definitions are different: evaluating a definition does not produce a value, but unlike an expression, it may introduce a new global variable or a new function. New variables are added to ξ and new functions are added to ϕ , so the evaluation of a definition is described by a judgment of the form $\langle d, \xi, \phi \rangle \rightarrow \langle \xi', \phi' \rangle$. Below, this judgment is used in rules for *true* definitions, which are implemented by the `evaldef` function in Section 1.6.2. *Extended* definitions aren't formalized.

Variable definition

The definition `VAL( $x, e$ )` introduces a global variable x . It adds the binding $x \mapsto v$ to the global environment ξ' , where v is the result of evaluating e . A definition does

not appear in the body of a function, so e is evaluated without any formal parameters (ρ is $\{\}$). The **DEFINEGLOBAL** rule closely resembles the **GLOBALASSIGN** rule, but the **DEFINEGLOBAL** rule does not require $x \in \text{dom } \xi$. It is OK if x is already a global variable, in which case $\text{VAL}(x, e)$ behaves just like $\text{SET}(x, e)$.

$$\frac{\langle e, \xi, \phi, \{\} \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}{\langle \text{VAL}(x, e), \xi, \phi \rangle \rightarrow \langle \xi' \{x \mapsto v\}, \phi \rangle} \quad (\text{DEFINEGLOBAL})$$

An imperative core

38

Function definition

The definition **DEFINE**($f, \langle x_1, \dots, x_n \rangle, e$) introduces a function f . The function is represented as **USER**($\langle x_1, \dots, x_n \rangle, e$), where $x_1, \dots, x_n$ are the names of the formal parameters and e is the body. No two parameters may have the same name.

$$\frac{x_1, \dots, x_n \text{ all distinct}}{\langle \text{DEFINE}(f, \langle x_1, \dots, x_n \rangle, e), \xi, \phi \rangle \rightarrow \langle \xi, \phi \{f \mapsto \text{USER}(\langle x_1, \dots, x_n \rangle, e)\} \rangle} \quad (\text{DEFINEFUNCTION})$$

Top-level expression

An expression e , when entered at the read-eval-print loop, also serves as a definition. This “definition” does not introduce any new names into ξ or ϕ ; e is evaluated for its side effect, and its value is stored in the global variable **it**. Therefore, evaluating an expression as a definition can modify the global-variable environment ξ but not the function environment ϕ .

$$\frac{\langle e, \xi, \phi, \{\} \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}{\langle \text{EXP}(e), \xi, \phi \rangle \rightarrow \langle \xi' \{\text{it} \mapsto v\}, \phi \rangle} \quad (\text{EVALEXP})$$

Evaluating the “definition” $\text{EXP}(e)$ has exactly the same effects as evaluating the more conventional definition $\text{VAL}(\text{it}, e)$.

Extended definitions

As mentioned in the sidebar on page 24, Impcore’s extended-definition forms **use**, **check-expect**, **check-assert**, and **check-error** aren’t specified formally. Evaluating **use** evaluates the definitions contained in the file named by the **use**, then runs that file’s unit tests. And evaluating a **check-expect**, a **check-assert**, or a **check-error** remembers a unit test. Specifying **use** formally would require a model of files with names and contents. Specifying unit tests would require adding a set of pending unit tests to our abstract machine, plus judgments and inference rules that would describe what it means for a test to succeed or fail. Such things would distract us from what the semantics is meant to do: help us understand and compare programming languages.

To understand these issues for yourself, try designing rules for extended definitions. Start by defining the success or failure of **check-expect** (Exercise 17) or the success of **check-error** (Exercise 21). To specify **check-error**, you will need to design a proof system that says what it means for evaluation to halt with an error.

1.6 THE INTERPRETER

A bridge language like Impcore is meant to be small enough to learn, small enough to specify, small enough to implement, and yet big enough to write interesting programs in. (Or in the case of Impcore, not quite big enough.) To write programs

requires an implementation, but why talk about it? Why not bury the implementation in a repository somewhere? Because the implementation of an interpreter can illustrate a language and its semantics in a way that nothing else can. And when you want to experiment with alternative language designs, you can build on or change my code. To make such experiments possible, I can't just hand you the code; I have to explain it. But I can't explain all of it—the explanations would add over 300 pages to this book. In this chapter, I explain just the most important parts.

A language is embodied by the data structures for its crucial abstractions (environments and abstract-syntax trees) and by the functions that evaluate expressions and definitions. They are all explained in this chapter. The crucial code is built on top of infrastructure: error handling, parsing, printing, test reporting, and so on. That infrastructure is lovingly described in a Supplement to this book, which is available from `build-prove-compare.net`. The interpreter requires only a standard C library, and with that and the Supplement, you can understand as much or as little as you wish.

The code is presented using the Noweb system for *literate programming*. Noweb extracts code directly from the text, so the code in the book is the code that runs. Noweb splits code into named “code chunks,” which are surrounded by textual explanations. The code chunks are written in an order designed to support good explanations, not the order dictated by a C compiler.

Code chunks can mix source code with references to other chunks. References are italicized in angle brackets, as in *(evaluate e-→ifx and return the result 49c)*. The label “49c” shows where to find the definition: the number identifies a page, and when the page contains more than one chunk, each chunk gets its own lower-case letter, which is appended to the page number. The label also appears on the first line of the definition, in bold. Each chunk definition is shown using the $\equiv$ sign. A definition can be continued in a later chunk; Noweb concatenates the contents of all definitions of the same chunk. A definition that continues a previous definition is showing using the $+\equiv$ sign in place of the $\equiv$ sign. When a chunk's definition is continued, the right margin displays pointers to the previous and next definitions, written “◁48a” and “S296a▷.” The notation “(48b)” shows where a chunk is used.

To help you find relevant chunks, Noweb provides a *mini-index* in the margin of each right-hand page. For example, the mini-index on page 49 reveals that function `bindval` is defined in chunk 45b on page 45. It also reveals that type `Exp` is not defined by hand-written code; it is generated automatically. In any mini-index, $\mathcal{A}$ stands for automatically generated code, and $\mathcal{B}$ stands for a basis function from C's standard library. And $\mathcal{P}$, which is used from Chapter 2 onward, stands for a *primitive* function that is defined in an interpreter.

In most chapters, Noweb's information is supplemented by a table that relates semantics, concepts, and code, like Table 1.5 on the next page. This table will help you learn the important parts of the code, which all relate to metavariables and math symbols from Section 1.5.

The code will be clearer to you if you know my programming conventions. For example, when I introduce a new type, I use `typedef` to give it a name that begins with a capital letter, like `Name`, or `Exp`, or `Def`. The representation of such a type is often *exposed*, in which case you get to see all of the type's definition, and you get access to fields of structures and so on. A type whose representation is exposed is called *manifest*; for example, types `Exp` and `Def` are manifest. A type might also be *abstract*, in which case you *can't* get at its representation. In C, an abstract type is always a pointer to a named `struct` whose fields are not specified. For example, type `Name` is abstract; you can store a `Name` in a field or a variable, and you can pass a `Name` to a function, but you can't look inside a `Name` to see how it is represented. (Strictly speaking, *you* can see everything; as explained in Chapter 9, it is your client code, and mine, that cannot see the representations of abstract types.)

Table 1.5: Correspondence between Impcore semantics and code

<i>Semantics</i>	<i>Concept</i>	<i>Interpreter</i>
d	True definition	Def (page 42)
e	Expression	Exp (page 42)
x, f	Name	Name (page 43)
v	Value	Value (page 43)
USER($\dots$)	Function	Userfun (page 42), Func (page 44)
PRIMITIVE($\oplus$)	Function	Name (page 43), Func (page 44)
ξ, ρ	Value environment	Valenv (page 44)
ϕ	Function environment	Funenv (page 44)
$\langle e, \xi, \phi, \rho \rangle \Downarrow$ $\langle v, \xi', \phi, \rho' \rangle$	Expression evaluation	eval(e, ξ, ϕ, ρ) = v , with ξ and ρ updated to ξ' and ρ' (page 48)
$\langle d, \xi, \phi \rangle \rightarrow \langle \xi', \rho' \rangle$	Definition evaluation	evaldef($e, \xi, \phi, \text{echo}$) updates ξ to ξ' and ϕ to ϕ' (page 53)
$x \in \text{dom } \rho$	Definedness	isvalbound (page 45)
$f \in \text{dom } \phi$	Definedness	isfunbound (page 45)
$\rho(x), \xi(x)$	Lookup	fetchval (page 44)
$\phi(f)$	Lookup	fetchfun (page 44)
$\rho\{x \mapsto v\}$	Binding	bindval (page 45)
$\phi\{f \mapsto \dots\}$	Binding	bindfun (page 45)

I write the names of functions using lowercase letters only, except for some automatically generated functions used to build lists.

To the degree that C permits, I distinguish interfaces from implementations. An interface typically includes some or all of these elements:

- Types, which may be manifest or abstract
- Invariants of manifest types, if any
- Prototypes of functions
- Documentation explaining how the types and functions should be used

An *atypical* interface might also include declarations of global variables, macros, or other arcana. But no interface, typical or atypical, ever includes the implementations of its functions.

Interface documentation explains not only how to use functions but also what happens when a function is used incorrectly; in particular, it explains who is responsible for detecting or avoiding an error. A *checked run-time error* is a mistake that the implementation guarantees to detect; a typical example would be passing a NULL pointer to a function. The implementation need not *recover* from a checked error, and indeed, many of my implementations simply halt with assertion failures.

An *unchecked* run-time error is more insidious; this is a mistake that it is up to the C programmer to avoid. If client code causes an unchecked run-time error, the implementation provides no guarantees; anything can happen. Unchecked run-time errors are part of the price we pay for programming in C.

Once you've read and understood an interface, you should be able to use its functions without needing to look at their implementations. But of course the cru-

A language in which all errors are checked is called *safe*. Safety is usually implemented by a combination of compile-time and run-time checking. Popular safe languages include Awk, C#, Haskell, Go, Java, JavaScript, Lua, ML, Perl, Python, Ruby, Rust, Scheme, and Smalltalk. A safe language might be characterized by saying that “there are no unexplained core dumps”; a program that halts always issues an informative error message.

A language that permits unchecked errors is called *unsafe*. Unsafe languages put an extra burden on the programmer, but they provide extra expressive power. This extra power is needed to write things like garbage collectors and device drivers; systems programming languages, like Bliss and C, have historically been unsafe. C++ is an anomaly: it is ostensibly intended for high-level problem-solving, but it is nevertheless unsafe.

A few well-designed systems-programming languages are safe by default, but have unsafe features that can be turned on explicitly at need, usually by a keyword UNSAFE. The best known of these may be Cedar and Modula-3.

cial implementations, of functions like `eval` and `evaldef`, are intended for you to look at. When you look at them, you’ll see that they respect these conventions:

- Within reason, each local variable is declared in the region in which it is used; local variables typically don’t scope over an entire function definition.
- When possible, each variable is initialized where it is declared, making its declaration resemble a `val` definition in Impcore.

On to the code! The presentation begins with the interfaces in Section 1.6.1. These interfaces include not only the interfaces associated with the evaluator, but also some interfaces associated with general-purpose utility code from the Supplement (Appendix F). These interfaces are necessary building blocks, but they aren’t the main event; the main event is the implementation of Impcore’s operational semantics in Section 1.6.2, which starts on page 48.

1.6.1 Interfaces

Everything comes together in the evaluator, which implements the operational semantics. That evaluator is supported by interfaces for the central structures of a programming language: syntax, names, values, environments, and lists thereof. It is also supported by interfaces used for printing and for reporting errors.

Interface to abstract syntax: Manifest types and creator functions

The abstract-syntax interface exposes the representations of definitions, expressions, and so on. It also provides convenient *creator functions* which are used to build abstract-syntax trees.

An abstract-syntax tree is a value of a *sum type*; such a type is also called a *discriminated-union type*. A sum type specifies a list of alternative forms; every value of the type takes the form of one of the alternatives. Sum types play an essential role in symbolic computing, but they are not directly supported in C. C provides only an *undiscriminated union*, sometimes also called an “unsafe” union. A C union specifies a list of alternative forms, called “members,” but from the union alone, you

can't tell what form a value or variable is intended to have. To represent a sum type in C requires not only a union but also a *discriminant* (a “tag”), which says which form of the union is in use. In this book, the tag and the union are placed together in a C struct. The tag is called `alt` (for “alternative”), and the union is unnamed. A tag is referred to by name, as in `e->alt`, and in the union, each alternative is referred to by name, as in `e->var` or `d->define`. An alternative may itself have multiple named parts; for example, if `d` represents a `val` definition, the name of the variable being defined is `d->val.name`, and the expression that gives it its initial value is `d->val.exp`.

The name of each alternative appears in two places: in an enumeration type that defines the possible values of `alt`, and in the union type, which lists the possible forms of the union. And in keeping with common C practice, each enumeration literal is written using all capital letters, and each member of the union is written in lower case.

Our first complete example is the true definition `Def`, a simplified version of which appears in chunk *(simplified example of abstract syntax for Impcore 27d)*. A `Def` is a sum type with three alternatives, called `VAL`, `EXP`, and `DEFINE`.

42a. *(type and structure definitions for Impcore 42a)* $\equiv$ 44c>

```
typedef struct Userfun Userfun;
struct Userfun { Namelist formals; Exp body; };

typedef struct Def *Def;
typedef enum { VAL, EXP, DEFINE } Default;
struct Def {
    Default alt;
    union {
        struct { Name name; Exp exp; } val;
        Exp exp;
        struct { Name name; Userfun userfun; } define;
    } ;
};
```

Such type definitions and creator functions are hard to write and maintain by hand. So I generate them automatically, using an ML program that appears in Appendix J. That program generates the `Def` type and associated functions from the following descriptions:

42b. *(definition.t 42b)* $\equiv$

```
Userfun = (Namelist formals, Exp body)
Def*    = VAL      (Name name, Exp exp)
        | EXP      (Exp)
        | DEFINE   (Name name, Userfun userfun)
```

A valid `Userfun` satisfies the invariant that the names in `formals` are all distinct.

The abstract syntax for `Exp`, which you might wish to compare with the concrete syntax given for *exp* on page 18, is described as follows:

42c. *(exp.t 42c)* $\equiv$

```
Exp* = LITERAL (Value)
     | VAR      (Name)
     | SET      (Name name, Exp exp)
     | IFX      (Exp cond, Exp truex, Exp falsex)
     | WHILEX   (Exp cond, Exp exp)
     | BEGIN    (Explist)
     | APPLY    (Name name, Explist actuals)
```

The descriptions above⁴ are slightly elaborated versions of *(simplified example of abstract syntax for Impcore 27d)*. Similar descriptions are used for much of the C code in this book.

True definitions and expressions are the essential elements of abstract syntax; once you understand how they work, you will be ready to connect the operational semantics and the code. Impcore's *extended* definitions, including unit tests, are described in the Supplement.

Interface to names: An abstract type

Programs are full of names. To make it easy to compare names and look them up in tables, I define an abstract type to represent them. Although each name is built from a string, the abstract type hides the string and its characters. Unlike C strings, names are immutable, and two names are equal if and only if they are the same pointer.

43a. *(shared type definitions 43a)* ≡ (S295a)

```
typedef struct Name *Name;
typedef struct Namelist *Namelist; // list of Name
```

A name may be built from a string or converted to a string.

43b. *(shared function prototypes 43b)* ≡ (S295a) 46b▷

```
Name strtoname(const char *s);
const char *nametostr(Name x);
```

These functions satisfy the following algebraic laws:

```
strcmp(s, nametostr(strtoname(s))) == 0
strcmp(s, t) == 0 if and only if strtoname(s) == strtoname(t)
```

The first law says if you build a name from a string, `nametostr` returns a copy of your original string. The second law says you can compare names using pointer equality.

Because `nametostr` returns a string of type `const char*`, a client of `nametostr` cannot modify that string without subverting the type system. Modification of the string is an unchecked run-time error. New values of type `Name*` should be created only by calling `strtoname`; to do so by casting other pointers is a subversion of the type system and an unchecked run-time error.

Interface to values

The value interface defines the type of value that an expression may evaluate to. In Impcore, that is always a 32-bit integer. A `Valuelist` is a list of `Values`.

43c. *(type definitions for Impcore 43c)* ≡ (S295a) 44a▷

```
typedef int32_t Value;
typedef struct Valuelist *Valuelist; // list of Value
```

⁴The alternatives for `if` and `while` are named `IFX` and `WHILEX`, not `IF` and `WHILE`. Why? Because corresponding to each alternative, there is a field of a union that uses the same name in *lower* case. For example, if `e` is a `LITERAL` expression, the literal `Value` is found in field `e->literal`. But a structure field can't be named `if` or `while`, because the names `if` and `while` are *reserved words*—they may be used only to mark C syntax. So I call these alternatives `IFX` and `WHILEX`, which I encourage you to think of as “if-expression” and “while-expression.” For similar reasons, the two branches of the `IFX` are called `truex` and `falsex`, not `true` and `false`. And in Chapter 2, you'll see `LETX` and `LAMBDAx` instead of `LET` and `LAMBDA`, so that I can write an interpreter for μ Scheme in μ Scheme.

In the Impcore interpreter, the type “function” is another sum type. This type specifies two alternatives: user-defined functions and primitive functions. Following the operational semantics, which represents a user-defined function as $\text{USER}(\langle x_1, \dots, x_n \rangle, e)$, a user-defined function is represented as a pair containing formals and body. A primitive is represented by its name.

44a. *(type definitions for Impcore 43c)* $\vdash \equiv$ (S295a) $\triangleleft 43c \ 44e \triangleright$

```
typedef struct Funclist *Funclist; // list of Func
```

44b. *(fun.t 44b)* $\equiv$

```
Func = USERDEF    (Userfun)
      | PRIMITIVE (Name)
```

This description is used to generate these type and structure definitions:

44c. *(type and structure definitions for Impcore 42a)* $\vdash \equiv$ $\triangleleft 42a$

```
typedef struct Func Func;
typedef enum { USERDEF, PRIMITIVE } Funcalt;
struct Func { Funcalt alt; union { Userfun userdef; Name primitive; } ; };
```

Also generated automatically are these prototypes for creator functions.

44d. *(function prototypes for Impcore 44d)* $\equiv$ (S295a) $\triangleleft 44f \triangleright$

```
Func mkUserdef(Userfun userdef);
Func mkPrimitive(Name primitive);
```

Interface to environments: More abstract types

In the operational semantics, the environments ρ and ξ hold values, and the environment ϕ holds functions. Each kind of environment has its own representation.⁵

44e. *(type definitions for Impcore 43c)* $\vdash \equiv$ (S295a) $\triangleleft 44a$

```
typedef struct Valenv *Valenv;
typedef struct Funenv *Funenv;
```

A new environment may be created by passing a list of names and a list of associated values or function definitions to `mkValenv` or `mkFunenv`. For example, calling `mkValenv( $\langle x_1, \dots, x_n \rangle, \langle v_1, \dots, v_n \rangle$ )` returns $\{x_1 \mapsto v_1, \dots, x_n \mapsto v_n\}$. Passing lists of different lengths is a checked run-time error.

44f. *(function prototypes for Impcore 44d)* $\vdash \equiv$ (S295a) $\triangleleft 44d \ 44g \triangleright$

```
Valenv mkValenv(Namelist vars, Valuelist vals);
Funenv mkFunenv(Namelist vars, Funclist defs);
```

A value or a function definition is retrieved using `fetchval` or `fetchfun`. In the operational semantics, the lookup `fetchval( $x, \rho$ )` is simply $\rho(x)$.

44g. *(function prototypes for Impcore 44d)* $\vdash \equiv$ (S295a) $\triangleleft 44f \ 45a \triangleright$

```
Value fetchval(Name name, Valenv env);
Func  fetchfun(Name name, Funenv env);
```

⁵By defining one C type for environments that hold a Value and another for environments that hold a Func, I ensure type safety—but at the cost of having to write two essentially identical versions of each function. In C, the only alternative is to define a single C type for environments, which would hold a void* pointer, which would then be cast to a Value* or Func* as needed. This choice duplicates no code, but it is unsafe; if we accidentally put a Value* in an environment intended to hold a Func*, it is an error that neither the C compiler nor the run-time system can detect. In the interests of safety, I duplicate code. Chapter 5 shows how in another implementation language, ML, we can use *polymorphism* to achieve type safety without duplicating code. Similar results can be obtained using C++, Java, and other languages.

If the given name is not bound in the environment, calling `fetchval` or `fetchfun` is a checked run-time error. To ensure that fetching is safe, first call `isvalbound` or `isfunbound`; these functions return 1 if the given name is in the environment, and 0 otherwise. Formally, `isvalbound( $x, \rho$ )` is written $x \in \text{dom } \rho$.

45a. *(function prototypes for Impcore 44d)* $\vdash \equiv$ (S295a) $\triangleleft 44g \ 45b \triangleright$

```
bool isvalbound(Name name, Valenv env);
bool isfunbound(Name name, Funenv env);
```

To add new bindings to an environment, use `bindval` and `bindfun`. Unlike the previous six functions, `bindval` and `bindfun` *mutate* their argument environments, replacing the old bindings with new ones. Calling `bindval( $x, v, \rho$ )` is equivalent to performing the assignment $\rho := \rho\{x \mapsto v\}$. Because ρ is a *mutable* abstraction, modifications to the environment are visible to whatever code calls `bindval`.

45b. *(function prototypes for Impcore 44d)* $\vdash \equiv$ (S295a) $\triangleleft 45a \ 45c \triangleright$

```
void bindval(Name name, Value val, Valenv env);
void bindfun(Name name, Func fun, Funenv env);
```

These functions can be used to replace existing bindings or to add new ones.

Interface to the evaluator

The evaluator works with abstract syntax and values, whose representations are exposed, and with names and environments, whose representations are not exposed. Its interface exports functions `eval` and `evaldef`, which evaluate expressions and true definitions, respectively. (Extended definitions are evaluated by function `readevalprint`, which is described in the Supplement.) Function `eval` implements the $\Downarrow$ relation in our operational semantics. For example, `eval( $e, \xi, \phi, \rho$ )` finds a v, ξ' , and ρ' such that $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle$, assigns $\rho := \rho'$ and $\xi := \xi'$, and returns v . Function `evaldef` similarly implements the $\rightarrow$ relation.

45c. *(function prototypes for Impcore 44d)* $\vdash \equiv$ (S295a) $\triangleleft 45b \triangleright$

```
Value eval (Exp e, Valenv globals, Funenv functions, Valenv formals);
void evaldef(Def d, Valenv globals, Funenv functions, Echo echo_level);
```

Just as the forms of the evaluation judgments tell us something about the operational semantics, the types of the evaluation functions tell us something about the implementation. The result types confirm that evaluating an `Exp` produces a value but evaluating a `Def` does not. Both kinds of evaluations can have side effects on environments. Finally, the `echo_level` parameter, which has no counterpart in the semantics, controls printing: when `echo_level` is `ECHOING`, `evaldef` prints the values and names of top-level expressions and functions. When `echo_level` is `NOT_ECHOING`, `evaldef` does not print.

Interface to lists

The evaluator's data structures include many lists: names, values, functions, expressions, and unit tests are all placed in lists. For safety, each list has its own type: `Namelist`, `Valuelist`, `Funclist`, `Explist`, and `UnitTestlist`. Each of these types is recursive; a list is either empty or is a pointer to a pair (`hd`, `tl`), where `hd` is the first element of the list and `tl` is the rest of the list. An empty list is represented by a null pointer. Each list type is defined in the same way, as in this example:

45d. *(example structure definitions for Impcore 45d)* $\equiv$

```
struct Explist {
    Exp hd;
    struct Explist *tl;
};
```

type Def	$\mathcal{A}$
type Echo	S293f
type Exp	$\mathcal{A}$
type Func	$\mathcal{A}$
type Name	43a
type Namelist	43a
type Userfun	$\mathcal{A}$
type Value	43c
type Valuelist	43c

The type definitions are generated by a Lua script, which searches header files for lines of the form

```
typedef struct Foo *Foo; // list of Foo
```

For each type of list, the script also generates a length function, an extractor, a creator function, and a print function. These functions are named `lengthTL`, `nthTL`, `mkTL`, and `printTL`, where *T* is the first letter of the list type. The length of the NULL list is zero; the length of any other list is the number of its elements. Elements are numbered from zero, and asking for `nthTL(xs, n)` when $n \geq \text{lengthTL}(xs)$ is a checked run-time error. Calling `mkTL` creates a fresh list with the new element at the head; it does not mutate the old list.

The list functions have prototypes like these:

46a. *(example function prototypes for Impcore 46a)* $\equiv$

```
int    lengthEL(Explist es);
Exp    nthEL    (Explist es, unsigned n);
Explist mkEL    (Exp e, Explist es);
Explist popEL   (Explist es);
```

Definitions and function prototypes for all the list types can be found in the interpreter's `all.h` file. Because of the repetition, this code is tedious to read, but generating the code automatically makes the tedium bearable. And ML's polymorphism enables a simpler solution (Chapter 5).

Interface to infrastructure: Printing

After evaluating a definition, the interpreter prints a name or a value. And when an error occurs, the interpreter may need to print a faulty expression or definition. Strings and numbers can easily be printed using `printf`, but expressions and definitions can't. So instead, the interpreter uses functions `print` and `fprint`, which replace `printf` and `fprintf`. These functions, which are defined in the Supplement, support direct printing of Exps, Defs, Names, and so on.

46b. *(shared function prototypes 43b)* $\equiv$ (S295a) <43b 47a>

```
void print (const char *fmt, ...); // print to standard output
void fprint(FILE *output, const char *fmt, ...); // print to given file
```

By design, `print` and `fprint` resemble `printf` and `fprintf`: the `fmt` parameter is a “format string” that contains “conversion specifications.” Our conversion specifications are like those used by `printf`, but much simpler. A conversion specification is two characters: a percent sign followed by a character like `d` or `s`, which is called a *conversion specifier*. Unlike standard conversion specifications, ours don't contain minus signs, numbers, or dots. The ones used in the Impcore interpreter are shown here in Table 1.6. By convention, lowercase specifiers print individual values; uppercase specifiers print lists. Most specifiers are named for the initial letter of what they print, but the specifier for a `Def` must not be `%d`: the `%d` is too firmly established as a specifier for printing decimal integers. Instead, `Def` is specified by `%t`, for “top level,” which is where a `Def` appears.

Functions `print` and `fprint` are *unsafe*; if you pass an argument that is not consistent with the corresponding conversion specifier, it is an *unchecked* run-time error.

Interface to infrastructure: Error handling

When it encounters a fault, the Impcore interpreter complains and recovers by calling a function in an error-handling interface. In general, a fault occurs whenever

Table 1.6: Conversion specifiers for impcore

%%	Print a percent sign
%d	Print an integer in decimal format
%e	Print an Exp
%f	Print a Func
%E	Print an Explist (list of Exp)
%n	Print a Name
%N	Print a Namelist (list of Name)
%p	Print a Par (see Appendix K)
%P	Print a Parlist (list of Par)
%s	Print a char* (string)
%t	Print a Def (t stands for “top level”, which is where definitions appear)
%v	Print a Value
%V	Print a Valuelist (list of Value)

a program is ill formed, ill typed, or ill behaved, but Impcore has no static type system, so faults are triggered only by ill-formed and ill-behaved programs:

- When it detects an ill-formed program during parsing, the interpreter signals a *syntax error* by calling `synerror`.
- When it detects an ill-behaved program at run time, the interpreter signals a *run-time error* by calling `runerror`.

Before initiating error recovery, each error-signaling function prints a message. For that reason, an error-signaling function’s interface resembles `print`. But because different information is available at parse time and at run time, `synerror` and `runerror` have different interfaces.

The simpler of the two is `runerror`. During normal operation, `runerror` prints to standard error and then `longjmp`s to `errorjmp`.

47a. *(shared function prototypes 43b)* +≡ (S295a) <46b 47b>

```
void runerror (const char *fmt, ...);
extern jmp_buf errorjmp;           // longjmp here on error
```

During unit testing, `runerror` operates in *testing* mode, and it behaves a little differently (Section F.5.1, page S182).

Function `synerror` is like `runerror`, except that before its format string, it takes an argument of type `Sourceloc`, which tracks the source-code location being read at the time of the error. The location can be printed as part of the error message.

47b. *(shared function prototypes 43b)* +≡ (S295a) <47a 47c>

```
void synerror (Sourceloc src, const char *fmt, ...);
```

type Exp $\mathcal{A}$
type Sourceloc
S293h

Error *handling*, as opposed to error signaling, is implemented by calling `setjmp` on `errorjmp`. Function `setjmp` must be called before any error-signaling function. It is an unchecked run-time error to call `runerror` or `synerror` except when a `setjmp` involving `errorjmp` is active on the C call stack.

One common run-time error is that an Impcore function is called with the wrong number of arguments. That error is detected by function `checkargc`. Its parameter `e` holds the call in which the error might occur.

47c. *(shared function prototypes 43b)* +≡ (S295a) <47b>

```
void checkargc(Exp e, int expected, int actual);
```

1.6.2 Implementation of the evaluator

The evaluator implements Impcore's semantics. It comprises functions `eval` and `evaldef`, which evaluate expressions and true definitions.

Evaluating expressions

Function `eval` implements the $\Downarrow$ relation from the operational semantics. Calling `eval( $e, \xi, \phi, \rho$ )` finds a v, ξ' , and ρ' such that $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle$, assigns $\rho := \rho'$ and $\xi := \xi'$, and returns v . Because Greek letters aren't customary in C code, I use these English names:

ξ globals
 ϕ functions
 ρ formals

Function `eval` is mutually recursive with a private helper function, `evallist`:

48a. $\langle \text{eval.c 48a} \rangle \equiv$ 48b $\triangleright$

```
static Valuelist evallist(Explist es, Valenv globals, Funenv functions,
                          Valenv formals);
```

Evaluation of an expression e begins by discovering its syntactic form, using a switch on the tag `e->alt`.

48b. $\langle \text{eval.c 48a} \rangle + \equiv$ $\triangleleft$ 48a 51b $\triangleright$

```
Value eval(Exp e, Valenv globals, Funenv functions, Valenv formals) {
    switch (e->alt) {
        case LITERAL:  $\langle \text{evaluate e->literal and return the result 48c} \rangle$ 
        case VAR:       $\langle \text{evaluate e->var and return the result 49a} \rangle$ 
        case SET:       $\langle \text{evaluate e->set and return the result 49b} \rangle$ 
        case IFX:       $\langle \text{evaluate e->ifx and return the result 49c} \rangle$ 
        case WHILEX:    $\langle \text{evaluate e->whilex and return the result 50a} \rangle$ 
        case BEGIN:     $\langle \text{evaluate e->begin and return the result 50b} \rangle$ 
        case APPLY:     $\langle \text{evaluate e->apply and return the result 50c} \rangle$ 
    }
    assert(0);
}
```

The assertion at the end of `eval` might seem superfluous, but it isn't; it helps protect me, and you, from mistakes, and it convinces the C compiler that every possible case is covered.

Function `eval` proceeds by case analysis over the syntactic forms of `Exp`. Each case is written in consultation with the operational semantics: for each syntactic form, `eval` implements the rules that have the form on the left-hand sides of their conclusions.

The `LITERAL` form appears in the conclusion of just one rule.

$$\frac{}{\langle \text{LITERAL}(v), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi, \phi, \rho \rangle} \quad (\text{LITERAL})$$

The implementation returns the literal value.

48c. $\langle \text{evaluate e->literal and return the result 48c} \rangle \equiv$ (48b)

```
return e->literal;
```

The `VAR` form appears in the conclusions of two rules.

$$\frac{x \in \text{dom } \rho}{\langle \text{VAR}(x), \xi, \phi, \rho \rangle \Downarrow \langle \rho(x), \xi, \phi, \rho \rangle} \quad (\text{FORMALVAR})$$

$$\frac{x \notin \text{dom } \rho \quad x \in \text{dom } \xi}{\langle \text{VAR}(x), \xi, \phi, \rho \rangle \Downarrow \langle \xi(x), \xi, \phi, \rho \rangle} \quad (\text{GLOBALVAR})$$

A rule can be used only if its premises hold, so the interpreter checks $x \in \text{dom } \rho$, which is implemented by calling `isvalbound(e->var, formals)`. If $x \notin \text{dom } \rho$ and $x \notin \text{dom } \xi$, the operational semantics gets stuck—so the interpreter issues an error message. Less formally, the interpreter looks for x in the formal-parameter environment first, then the global environment.

49a. *(evaluate e->var and return the result 49a)* $\equiv$ (48b)

```
if (isvalbound(e->var, formals))
    return fetchval(e->var, formals);
else if (isvalbound(e->var, globals))
    return fetchval(e->var, globals);
else
    runerror("unbound variable %n", e->var);
```

The call to `runerror` illustrates the convenience of the extensible printer; it uses `%n` to print a Name directly, without needing to convert the Name to a string.

The SET form is very similar. Again there are two rules, and again they are distinguished by testing $x \in \text{dom } \rho$.

$$\frac{x \in \text{dom } \rho \quad \langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}{\langle \text{SET}(x, e), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \{x \mapsto v\} \rangle} \quad (\text{FORMALASSIGN})$$

$$\frac{x \notin \text{dom } \rho \quad x \in \text{dom } \xi \quad \langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}{\langle \text{SET}(x, e), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi' \{x \mapsto v\}, \phi, \rho' \rangle} \quad (\text{GLOBALASSIGN})$$

Because both rules require the premise $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle$, the code evaluates e (`e->set.exp`) first, then puts its value in v .

49b. *(evaluate e->set and return the result 49b)* $\equiv$ (48b)

```
{
    Value v = eval(e->set.exp, globals, functions, formals);

    if (isvalbound(e->set.name, formals))
        bindval(e->set.name, v, formals);
    else if (isvalbound(e->set.name, globals))
        bindval(e->set.name, v, globals);
    else
        runerror("tried to set unbound variable %n in %e", e->set.name, e);
    return v;
}
```

The IF form appears in the conclusions of two rules.

$$\frac{\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v_1, \xi', \phi, \rho' \rangle \quad v_1 \neq 0 \quad \langle e_2, \xi', \phi, \rho' \rangle \Downarrow \langle v_2, \xi'', \phi, \rho'' \rangle}{\langle \text{IF}(e_1, e_2, e_3), \xi, \phi, \rho \rangle \Downarrow \langle v_2, \xi'', \phi, \rho'' \rangle} \quad (\text{IFTRUE})$$

$$\frac{\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v_1, \xi', \phi, \rho' \rangle \quad v_1 = 0 \quad \langle e_3, \xi', \phi, \rho' \rangle \Downarrow \langle v_3, \xi'', \phi, \rho'' \rangle}{\langle \text{IF}(e_1, e_2, e_3), \xi, \phi, \rho \rangle \Downarrow \langle v_3, \xi'', \phi, \rho'' \rangle} \quad (\text{IFFALSE})$$

Both rules have the same first premise: $\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v_1, \xi', \phi, \rho' \rangle$. To get v_1, ξ' , and ρ' , the code calls `eval(e->ifx.cond, globals, functions, formals)` recursively. This call may mutate the `globals` and `formals` environments, but regardless of whether $v_1 = 0$, the mutation is safe, because the third premises of *both* rules use the new environments ξ' and ρ' . Comparing v_1 with zero determines which rule should be used: the implementation ends with a recursive call to evaluate either e_2 (`e->ifx.true`) or e_3 (`e->ifx.false`).

49c. *(evaluate e->ifx and return the result 49c)* $\equiv$ (48b)

```
if (eval(e->ifx.cond, globals, functions, formals) != 0)
    return eval(e->ifx.true, globals, functions, formals);
else
    return eval(e->ifx.false, globals, functions, formals);
```

bindval	45b
eval	45c
type Exp	$\mathcal{A}$
type Explist	S292d
fetchval	44g
type Funenv	44e
isvalbound	45a
runerror	47a
type Valenv	44e
type Value	43c
type Valuelist	43c

The WHILE form appears in the conclusions of two rules.

1

$$\begin{array}{c}
 \frac{\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v_1, \xi', \phi, \rho' \rangle \quad v_1 \neq 0}{\langle e_2, \xi', \phi, \rho' \rangle \Downarrow \langle v_2, \xi'', \phi, \rho'' \rangle \quad \langle \text{WHILE}(e_1, e_2), \xi'', \phi, \rho'' \rangle \Downarrow \langle v_3, \xi''', \phi, \rho''' \rangle} \\
 \langle \text{WHILE}(e_1, e_2), \xi, \phi, \rho \rangle \Downarrow \langle v_3, \xi''', \phi, \rho''' \rangle \quad (\text{WHILEITERATE}) \\
 \frac{\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v_1, \xi', \phi, \rho' \rangle \quad v_1 = 0}{\langle \text{WHILE}(e_1, e_2), \xi, \phi, \rho \rangle \Downarrow \langle 0, \xi', \phi, \rho' \rangle} \quad (\text{WHILEEND})
 \end{array}$$

An imperative core

50

In the first rule, the premise $\langle \text{WHILE}(e_1, e_2), \xi'', \phi, \rho'' \rangle \Downarrow \langle v_3, \xi''', \phi, \rho''' \rangle$ could be implemented as a recursive call to `eval(e, ...)`. But `e` is always a while loop, so I have optimized the code by turning the recursion into iteration. This optimization prevents a long WHILE loop from overflowing the C stack.

50a. *⟨evaluate e→whilex and return the result 50a⟩* $\equiv$ (48b)

```
while (eval(e→whilex.cond, globals, functions, formals) != 0)
    eval(e→whilex.exp, globals, functions, formals);
return 0;
```

The BEGIN form appears in the conclusions of two rules.

$$\begin{array}{c}
 \overline{\langle \text{BEGIN}(), \xi, \phi, \rho \rangle \Downarrow \langle 0, \xi, \phi, \rho \rangle} \quad (\text{EMPTYBEGIN}) \\
 \langle e_1, \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v_1, \xi_1, \phi, \rho_1 \rangle \\
 \langle e_2, \xi_1, \phi, \rho_1 \rangle \Downarrow \langle v_2, \xi_2, \phi, \rho_2 \rangle \\
 \vdots \\
 \frac{\langle e_n, \xi_{n-1}, \phi, \rho_{n-1} \rangle \Downarrow \langle v_n, \xi_n, \phi, \rho_n \rangle}{\langle \text{BEGIN}(e_1, e_2, \dots, e_n), \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v_n, \xi_n, \phi, \rho_n \rangle} \quad (\text{BEGIN})
 \end{array}$$

A nonempty BEGIN is implemented by iterating over its subexpressions, leaving the last value in variable `lastval`. If `lastval` is initialized to zero, the same code also implements the empty BEGIN.

50b. *⟨evaluate e→begin and return the result 50b⟩* $\equiv$ (48b)

```
{
    Value lastval = 0;
    for (Explist es = e→begin; es; es = es→tl)
        lastval = eval(es→hd, globals, functions, formals);
    return lastval;
}
```

Function application appears in the conclusion of the APPLYUSER rule, and also in every rule that describes a primitive function. The rule to be implemented depends on the form of the function, which may be USERDEF or PRIMITIVE. Given a function named `f` (`e→apply.name`), the interpreter discovers its form by looking at $\phi(f)$, which it stores in local variable `f`.

50c. *⟨evaluate e→apply and return the result 50c⟩* $\equiv$ (48b)

```
{
    Func f;
    ⟨make f the function denoted by e→apply.name, or call runerror 51a⟩
    switch (f.alt) {
    case USERDEF:  ⟨apply f.userdef and return the result 51c⟩
    case PRIMITIVE: ⟨apply f.primitive and return the result 52a⟩
    default:       assert(0);
    }
}
```

If f is not defined as a function, the result is a run-time error.

51a. $\langle \text{make } f \text{ the function denoted by } e \rightarrow \text{apply.name, or call runerror 51a} \rangle \equiv$ (50c)

```
if (!isfunbound(e->apply.name, functions))
  runerror("call to undefined function %n in %e", e->apply.name, e);
f = fetchfun(e->apply.name, functions);
```

When f is a user-defined function, applying it has something in common with begin: arguments $e_1, \dots, e_n$ have to be evaluated. But where begin keeps only result v_n (in variable v in chunk 50b), function application keeps *all* the result values, which it binds into a new environment.

$$\begin{array}{c} \phi(f) = \text{USER}(\langle x_1, \dots, x_n \rangle, e) \\ x_1, \dots, x_n \text{ all distinct} \\ \langle e_1, \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v_1, \xi_1, \phi, \rho_1 \rangle \\ \vdots \\ \langle e_n, \xi_{n-1}, \phi, \rho_{n-1} \rangle \Downarrow \langle v_n, \xi_n, \phi, \rho_n \rangle \\ \frac{\langle e, \xi_n, \phi, \{x_1 \mapsto v_1, \dots, x_n \mapsto v_n\} \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}{\langle \text{APPLY}(f, e_1, \dots, e_n), \xi_0, \phi, \rho_0 \rangle \Downarrow \langle v, \xi', \phi, \rho_n \rangle} \quad (\text{APPLYUSER}) \end{array}$$

Values $v_1, \dots, v_n$, are returned by auxiliary function `evallist`, which is given $e_1, \dots, e_n$ along with ξ_0, ϕ , and ρ_0 . It evaluates $e_1, \dots, e_n$ in order, and it mutates the environments so that when it is finished, `globals` is ξ_n and `formals` is ρ_n . Finally, `evallist` returns the list $v_1, \dots, v_n$.

51b. $\langle \text{eval.c 48a} \rangle + \equiv$ $\langle 48b \text{ } 53b \rangle$

```
static Valuelist evallist(Explist es, Valenv globals, Funenv functions,
                          Valenv formals)
{
  if (es == NULL) {
    return NULL;
  } else {
    Value v = eval(es->hd, globals, functions, formals);
    return mkVL(v, evallist(es->tl, globals, functions, formals));
  }
}
```

The rules of `Impcore` require that `es->hd` be evaluated before `es->tl`. To ensure the correct order of evaluation, `eval(es->hd, ...)` and `evallist(es->tl, ...)` are called in separate C statements. Writing both calls as parameters to `mkVL` would not guarantee the correct order.

The `APPLYUSER` rule requires that the application have exactly as many arguments as f is expecting: the list of formal parameters and the list of argument expressions both have length n . But in general, they might be different: the formals $x_1, \dots, x_n$ are stored in `xs` and the actual values $v_1, \dots, v_m$ are stored in `vs`. When both are the same length, as confirmed by `checkargc`, they are used to create the fresh environment `mkValenv(xs, vs)`, which is $\{x_1 \mapsto v_1, \dots, x_n \mapsto v_n\}$. This environment is used to evaluate f 's body.

51c. $\langle \text{apply } f \text{.userdef and return the result 51c} \rangle \equiv$ (50c)

```
{
  Namelist xs = f.userdef.formals;
  Valuelist vs = evallist(e->apply.actuals, globals, functions, formals);
  checkargc(e, lengthNL(xs), lengthVL(vs));
  return eval(f.userdef.body, globals, functions, mkValenv(xs, vs));
}
```

Each primitive function is applied by code that is specialized to that primitive. A primitive is identified by comparing its name to a name made from a known

checkargc	47c
eval	45c
evallist	48a
type Explist	S292d
fetchfun	44g
formals	48b
type Func	A
functions	48b
type Funenv	44e
globals	48b
isfunbound	45a
lengthNL	A
lengthVL	A
mkValenv	44f
mkVL	A
type Namelist	
	43a
runerror	47a
type Valenv	44e
type Value	43c
type Valuelist	
	43c

string. More general techniques for implementing primitives, which are appropriate for larger languages, are shown in the Chapter 2 (page 154).

52a. $\langle \text{apply } f.\text{primitive and return the result } 52a \rangle \equiv$ (50c)

```

{
    Valuelist vs = evallist(e->apply.actuals, globals, functions, formals);
    if (f.primitive == strtoname("print"))
         $\langle \text{apply Impcore primitive print to vs and return } 52b \rangle$ 
    else if (f.primitive == strtoname("println"))
         $\langle \text{apply Impcore primitive println to vs and return } S299c \rangle$ 
    else if (f.primitive == strtoname("printu"))
         $\langle \text{apply Impcore primitive printu to vs and return } S299d \rangle$ 
    else
         $\langle \text{apply arithmetic primitive to vs and return } 52c \rangle$ 
}

```

Only print is implemented here; println and printu are in Appendix K.

52b. $\langle \text{apply Impcore primitive print to vs and return } 52b \rangle \equiv$ (52a)

```

{
    checkargc(e, 1, lengthVL(vs));
    Value v = nthVL(vs, 0);
    print("%v", v);
    return v;
}

```

Each arithmetic primitive expects exactly two arguments, which the code puts in C variables v and w. The characters of the primitive's name go in s.

52c. $\langle \text{apply arithmetic primitive to vs and return } 52c \rangle \equiv$ (52a)

```

{
    checkargc(e, 2, lengthVL(vs));
    Value v = nthVL(vs, 0);
    Value w = nthVL(vs, 1);
    const char *s = nametostr(f.primitive);
     $\langle \text{if operation s would overflow on v and w, call runerror } 53a \rangle$ 
     $\langle \text{return a function of v and w determined by s } 52d \rangle$ 
}

```

Ignoring the possibility of overflow, each Impcore primitive can be implemented by the corresponding operation in C. The primitive's name is synonymous with its first character, s[0].

52d. $\langle \text{return a function of v and w determined by s } 52d \rangle \equiv$ (52c)

```

assert(strlen(s) == 1);
switch (s[0]) {
case '<':    return v < w;
case '>':    return v > w;
case '=':   return v == w;
case '+':   return v + w;
case '-':   return v - w;
case '*':   return v * w;
case '/':   if (w == 0)
                runerror("division by zero in %e", e);
            return v / w;
default:    assert(0);
}

```

But the interpreter cannot ignore the possibility of overflow. The rules of Impcore are different from the rules of C, and if the result of an arithmetic operation does not fit in the range -2^{31} to 2^{31} , the operation causes a checked run-time error.

The error is detected and signaled by function `checkarith`, which is defined in Appendix F.

53a. $\langle \text{if operation } s \text{ would overflow on } v \text{ and } w, \text{ call } \text{runerror } 53a \rangle \equiv$ (52c)
`checkarith(s[0], v, w, 32);`

Evaluating true definitions

As noted on page 24, definitions are divided into two forms: The true definitions can differ in each language; Impcore's true definitions include `val` and `define`. The extended definitions are shared across languages; they include `use` and `check-expect`. True definitions have an operational semantics; extended definitions don't. And true definitions are evaluated by code that is explained here; extended definitions are evaluated by code in the Supplement.

The $\rightarrow$ relation on the true definitions is implemented by function `evaldef`. Calling `evaldef(d,  $\xi$ ,  $\phi$ , echo)` finds a ξ' and ϕ' such that $\langle d, \xi, \phi \rangle \rightarrow \langle \xi', \phi' \rangle$, and `evaldef` mutates the C representation of the environments so the global-variable environment becomes ξ' and the function environment becomes ϕ' . If `echo` is `ECHOING`, `evaldef` also prints the interpreter's response to the user's input. Printing the response is `evaldef`'s job because only `evaldef` can tell whether to print a value (for `EXP` and `VAL`) or a name (for `DEFINE`).

Just like `eval`, `evaldef` looks at the syntactic form of `d` and implements whatever rules have that form in their conclusions.

53b. $\langle \text{eval.c } 48a \rangle + \equiv$ <51b

```
void evaldef(Def d, Valenv globals, Funenv functions, Echo echo) {
    switch (d->alt) {
        case VAL:
             $\langle \text{evaluate } d \rightarrow \text{val, mutating globals } 53c \rangle$ 
            return;
        case EXP:
             $\langle \text{evaluate } d \rightarrow \text{exp and possibly print the result } 54a \rangle$ 
            return;
        case DEFINE:
             $\langle \text{evaluate } d \rightarrow \text{define, mutating functions } 54b \rangle$ 
            return;
    }
    assert(0);
}
```

A `VAL` form updates ξ .

$$\frac{\langle e, \xi, \phi, \{\} \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}{\langle \text{VAL}(x, e), \xi, \phi \rangle \rightarrow \langle \xi' \{x \mapsto v\}, \phi \rangle} \quad (\text{DEFINEGLOBAL})$$

The premise shows that value v and environment ξ' are obtained by calling `eval`. This call uses an empty environment as ρ . In the conclusion, the new environment ξ' is retained, and the value of the expression, v , is bound to x in it. Value v may also be printed.

53c. $\langle \text{evaluate } d \rightarrow \text{val, mutating globals } 53c \rangle \equiv$ (53b)

```
{
    Value v = eval(d->val.exp, globals, functions, mkValenv(NULL, NULL));
    bindval(d->val.name, v, globals);
    if (echo == ECHOING)
        print("%v\n", v);
}
```

§1.6
The interpreter
 53

<code>bindval</code>	45b
<code>checkargc</code>	47c
<code>checkarith</code>	S187b
<code>type Def</code>	$\mathcal{A}$
<code>type Echo</code>	S293f
<code>eval</code>	45c
<code>evallist</code>	48a
<code>formals</code>	48b
<code>functions</code>	48b
<code>type Funenv</code>	44e
<code>globals</code>	48b
<code>lengthVL</code>	$\mathcal{A}$
<code>mkValenv</code>	44f
<code>nametostr</code>	43b
<code>nthVL</code>	$\mathcal{A}$
<code>print</code>	S176d
<code>runerror</code>	47a
<code>strtoname</code>	43b
<code>type Valenv</code>	44e
<code>type Value</code>	43c
<code>type Valuelist</code>	43c

An EXP form also updates ξ , just as if it were a definition of it.

$$\frac{\langle e, \xi, \phi, \{\} \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}{\langle \text{EXP}(e), \xi, \phi \rangle \rightarrow \langle \xi' \{it \mapsto v\}, \phi \rangle} \quad (\text{EVALEXP})$$

1

54a. *(evaluate d→exp and possibly print the result 54a)* ≡ (53b)

```
{
    Value v = eval(d→exp, globals, functions, mkValenv(NULL, NULL));
    bindval(strtoname("it"), v, globals);
    if (echo == ECHOING)
        print("%v\n", v);
}
```

An imperative core

54

A DEFINE form updates ϕ . The implementation may print the name of the function being defined.

$$\frac{x_1, \dots, x_n \text{ all distinct}}{\langle \text{DEFINE}(f, \langle x_1, \dots, x_n \rangle, e), \xi, \phi \rangle \rightarrow \langle \xi, \phi \{f \mapsto \text{USER}(\langle x_1, \dots, x_n \rangle, e)\} \rangle} \quad (\text{DEFINEFUNCTION})$$

54b. *(evaluate d→define, mutating functions 54b)* ≡ (53b)

```
bindfun(d→define.name, mkUserdef(d→define.userfun), functions);
if (echo == ECHOING)
    print("%n\n", d→define.name);
```

The evaluator does not check to see that the $x_1, \dots, x_n$ are all distinct—the x_i 's are checked when the definition is parsed, by function `check_def_duplicates` in chunk S208e.

1.6.3 Implementation of environments

An environment is represented by a pair of lists; one holds names and the other holds the corresponding values. The lists have the same length. (A search tree or hash table would be enable faster search but would be more complicated.)

54c. *(env.c 54c)* ≡ 54d>

```
struct Valenv {
    Namelist xs;
    Valuelist vs;
    // invariant: lists have the same length
};
```

Given the representation, creating an environment is simple. To prevent the invariant from being violated, the code asserts that `xs` and `vs` have equal length.

54d. *(env.c 54c)* + ≡ <54c 55a>

```
Valenv mkValenv(Namelist xs, Valuelist vs) {
    Valenv env = malloc(sizeof(*env));
    assert(env != NULL);
    assert(lengthNL(xs) == lengthVL(vs));
    env->xs = xs;
    env->vs = vs;
    return env;
}
```

The list of names `xs` is searched by three functions: `fetchval`, `isvalbound`, and `bindval`. Behind the scenes, that search is implemented just once, in private function `findval`. Given a name `x`, it searches the environment. If it doesn't find `x`, it returns `NULL`. If it does find `x`, it returns a pointer to the *value* associated with `x`.

The pointer can be used to test for binding (`isvalbound`), to fetch a bound value (`fetchval`), or to change an existing binding (`bindval`).

55a. $\langle env.c\ 54c \rangle + \equiv$ $\langle 54d\ 55b \rangle$

```
static Value* findval(Name x, Valenv env) {
    Namelist xs;
    Valuelist vs;

    for (xs=env->xs, vs=env->vs; xs && vs; xs=xs->tl, vs=vs->tl)
        if (x == xs->hd)
            return &vs->hd;
    return NULL;
}
```

A name is bound if there is a value associated with it.

55b. $\langle env.c\ 54c \rangle + \equiv$ $\langle 55a\ 55c \rangle$

```
bool isvalbound(Name name, Valenv env) {
    return findval(name, env) != NULL;
}
```

A value is fetched through the pointer returned by `findval`, if any.

55c. $\langle env.c\ 54c \rangle + \equiv$ $\langle 55b\ 55d \rangle$

```
Value fetchval(Name name, Valenv env) {
    Value *vp = findval(name, env);
    assert(vp != NULL);
    return *vp;
}
```

A new binding could be added to an environment by inserting a new name and value at the beginning of `xs` and `vs`. But I can get away with an optimization. If $x \in \text{dom } \rho$, instead of extending ρ by making $\rho\{x \mapsto v\}$, I overwrite the old binding of x . This optimization is safe only because no program written in Impcore can tell that it is there. Proving that the optimization is safe requires reasoning about the rules of the operational semantics, which show that in any context where $\rho\{x \mapsto v\}$ appears, the old $\rho(x)$ can't affect any evaluations (Exercise 29 on page 85).

55d. $\langle env.c\ 54c \rangle + \equiv$ $\langle 55c \rangle$

```
void bindval(Name name, Value val, Valenv env) {
    Value *vp = findval(name, env);
    if (vp != NULL)
        *vp = val;           // safe optimization
    else {
        env->xs = mkNL(name, env->xs);
        env->vs = mkVL(val, env->vs);
    }
}
```

The code above implements *value* environments. To preserve type safety, the same data structure and operations are implemented a second time using `Func` instead of `Value`. That code appears in Appendix K.

§1.7
Operational
semantics
revisited: Proofs

55

bindfun	45b
bindval	45b
echo	53b
eval	45c
functions	53b
globals	53b
lengthNL	$\mathcal{A}$
lengthVL	$\mathcal{A}$
mkNL	$\mathcal{A}$
mkUserdef	44d
mkValenv	44f
mkVL	$\mathcal{A}$
type Name	43a
type Namelist	43a
print	S176d
strtoname	43b
type Valenv	44e
type Value	43c
type Valuelist	43c

1.7 OPERATIONAL SEMANTICS REVISITED: PROOFS

Calling $\text{eval}(e, \xi, \phi, \rho)$ is supposed to return a value v if and only if there is a *proof* of the judgment $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi, \phi, \rho \rangle$ using the rules from Section 1.5. Proofs in programming languages are not very interesting; or rather, they are interesting primarily when they are wrong. Like a bug in a program, a wrong proof tells you that you made a mistake (in your language design, not your code). And a good proof

doesn't have to be interesting, because it conveys *certainty*. A proof can guarantee that your program or your programming *language* does what you think it does.

1

1.7.1 Proofs about evaluation: Theory

A proof about the evaluation of code—that is, about a call to `eval` or `evaldef`—takes the form of a *derivation*. This form of proof, which is borrowed from formal logic, is a tree in which each node is an *instance* of an inference rule; an instance of a rule is obtained by substituting for the rule's metavariables. If every substitution is done consistently, and if, in the resulting derivation, every proposition holds, then the derivation is *valid*. A valid derivation justifies the judgment claimed in the conclusion of its root. These concepts are illustrated in the next few pages.

How do we write derivations?

A derivation is also called a *proof tree*; the root contains the conclusion, and each subtree is also a derivation. A derivation tree is written with its root at the bottom; as in a single rule, the conclusion of a derivation appears on the bottom, below a horizontal line. The leaf nodes appear at the top; each leaf node is an instance of an inference rule that has no evaluation judgments among its premises, like the `LITERAL` rule or the `FORMALVAR` rule (page 32). A leaf node corresponds to a computation in which `eval` returns a result without making a recursive call.

Each node in a derivation tree is obtained by *instantiating* a rule and then deriving that rule's premises. Instantiation may substitute for none, some, or all of a rule's metavariables. Substitution that replaces all metavariables, leaving only complete environments, syntax, and data, describes a single run of `eval`. For example, suppose a literal 83 is evaluated in a context where there are no global variables, no functions, and no formal parameters:

```
eval(mkLiteral(83),
      mkValenv(NULL, NULL), mkFunenv(NULL, NULL), mkValenv(NULL, NULL));
```

The resulting run can be described by an instance of the `LITERAL` rule. The rule appears in the semantics as follows:

$$\frac{}{\langle \text{LITERAL}(v), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi, \phi, \rho \rangle} \quad (\text{LITERAL})$$

The instance that describes the run is obtained by substituting 83 for v , $\{\}$ for ξ , $\{\}$ for ϕ , and $\{\}$ for ρ :

$$\frac{}{\langle \text{LITERAL}(83), \{\}, \{\}, \{\} \rangle \Downarrow \langle 83, \{\}, \{\}, \{\} \rangle}$$

Because the `LITERAL` rule has no premises above the line, this instance is a complete, valid derivation all by itself.

The preceding example is awfully specific. In practice, a literal 83 evaluates to 83 *regardless* of the presence of functions or variables—and the evaluation does not change the values of any variables. To prove that, I create a different instance of `LITERAL`, in which I substitute only for v , leaving ξ , ϕ , and ρ as they are written in the rule:

$$\frac{}{\langle \text{LITERAL}(83), \xi, \phi, \rho \rangle \Downarrow \langle 83, \xi, \phi, \rho \rangle}$$

This instance is also a complete, valid derivation, and it describes the evaluation of a literal 83 in any possible environment.

A complete derivation tree *ends* in a single rule, but if that rule has evaluation judgments above the line, like the APPLY rules, then each of those judgments—the premises—must be derived as well. A complete derivation follows this schema:

$$\boxed{\text{Derivation}} = \frac{\boxed{\text{Derivation of first premise}} \quad \dots \quad \boxed{\text{Derivation of last premise}}}{\boxed{\text{Conclusion}}}.$$

§1.7
Operational
semantics
revisited: Proofs
57

This schema assumes that every premise is justified by a derivation. In practice, only an evaluation judgment can be justified by a derivation. Other premises, like $x \in \text{dom } \rho$ or $\rho(x) = 3$, are justified by appealing to what we know about x and ρ .

As an example of a derivation tree with more than one node (which is wide enough to extend into the margin), I describe the evaluation of $(+ (* x x) (* y y))$, which computes the sum of two squares. It's evaluated in an environment where ρ binds x to 3 and y to 4.

$$\begin{array}{c} \text{FORMALVAR} \frac{x \in \text{dom } \rho \quad \rho(x) = 3}{\langle \text{VAR}(x), \xi, \phi, \rho \rangle \Downarrow \langle 3, \xi, \phi, \rho \rangle} \quad \text{FORMALVAR} \frac{x \in \text{dom } \rho \quad \rho(x) = 3}{\langle \text{VAR}(x), \xi, \phi, \rho \rangle \Downarrow \langle 3, \xi, \phi, \rho \rangle} \\ \text{APPLYMUL} \frac{\langle \text{VAR}(x), \xi, \phi, \rho \rangle \Downarrow \langle 3, \xi, \phi, \rho \rangle \quad \langle \text{VAR}(x), \xi, \phi, \rho \rangle \Downarrow \langle 3, \xi, \phi, \rho \rangle}{\langle \text{APPLY}(*, \text{VAR}(x), \text{VAR}(x)), \xi, \phi, \rho \rangle \Downarrow \langle 9, \xi, \phi, \rho \rangle} \quad \dots \\ \text{APPLYADD} \frac{\langle \text{APPLY}(*, \text{VAR}(x), \text{VAR}(x)), \xi, \phi, \rho \rangle \Downarrow \langle 9, \xi, \phi, \rho \rangle \quad \dots}{\langle \text{APPLY}(+, \text{APPLY}(*, \text{VAR}(x), \text{VAR}(x)), \text{APPLY}(*, \text{VAR}(y), \text{VAR}(y))), \xi, \phi, \rho \rangle \Downarrow \langle 25, \xi, \phi, \rho \rangle} \end{array}$$

Each node is labeled with the name of the rule to which it corresponds. Because derivation trees take so much space, I've elided the subtree that proves

$$\langle \text{APPLY}(*, \text{VAR}(y), \text{VAR}(y)), \xi, \phi, \rho \rangle \Downarrow \langle 16, \xi, \phi, \rho \rangle.$$

Derivation trees can get big, and to fit them into small spaces, we often take liberties with notation. For example, instead of writing abstract syntax like $\text{APPLY}(*, \text{VAR}(y), \text{VAR}(y))$, we can write concrete syntax like $(* y y)$. The resulting notation is easier to digest, but it is less obvious that each node is an instance of a semantic rule:

$$\begin{array}{c} \text{FORMALVAR} \frac{x \in \text{dom } \rho \quad \rho(x) = 3}{\langle x, \xi, \phi, \rho \rangle \Downarrow \langle 3, \xi, \phi, \rho \rangle} \quad \text{FORMALVAR} \frac{x \in \text{dom } \rho \quad \rho(x) = 3}{\langle x, \xi, \phi, \rho \rangle \Downarrow \langle 3, \xi, \phi, \rho \rangle} \\ \text{APPLYMUL} \frac{\langle x, \xi, \phi, \rho \rangle \Downarrow \langle 3, \xi, \phi, \rho \rangle \quad \langle x, \xi, \phi, \rho \rangle \Downarrow \langle 3, \xi, \phi, \rho \rangle}{\langle (* x x), \xi, \phi, \rho \rangle \Downarrow \langle 9, \xi, \phi, \rho \rangle} \quad \dots \\ \text{APPLYADD} \frac{\langle (* x x), \xi, \phi, \rho \rangle \Downarrow \langle 9, \xi, \phi, \rho \rangle \quad \dots}{\langle (+ (* x x) (* y y)), \xi, \phi, \rho \rangle \Downarrow \langle 25, \xi, \phi, \rho \rangle} \end{array}$$

Even if I use a smaller font and don't label the nodes, the full derivation tree sticks even further into the margin:

$$\frac{\frac{x \in \text{dom } \rho \quad \rho(x) = 3}{\langle x, \xi, \phi, \rho \rangle \Downarrow \langle 3, \xi, \phi, \rho \rangle} \quad \frac{x \in \text{dom } \rho \quad \rho(x) = 3}{\langle x, \xi, \phi, \rho \rangle \Downarrow \langle 3, \xi, \phi, \rho \rangle} \quad \frac{y \in \text{dom } \rho \quad \rho(y) = 4}{\langle y, \xi, \phi, \rho \rangle \Downarrow \langle 4, \xi, \phi, \rho \rangle} \quad \frac{y \in \text{dom } \rho \quad \rho(y) = 4}{\langle y, \xi, \phi, \rho \rangle \Downarrow \langle 4, \xi, \phi, \rho \rangle}}{\frac{\langle (* x x), \xi, \phi, \rho \rangle \Downarrow \langle 9, \xi, \phi, \rho \rangle \quad \langle (* y y), \xi, \phi, \rho \rangle \Downarrow \langle 16, \xi, \phi, \rho \rangle}{\langle (+ (* x x) (* y y)), \xi, \phi, \rho \rangle \Downarrow \langle 25, \xi, \phi, \rho \rangle}}.$$

What is a valid derivation?

A derivation is *valid* if every node is obtained by instantiating a rule and every premise is justified (either by a valid subderivation or by other means). To explain validity, it helps to name the rule that is instantiated at the root. A derivation $\mathcal{D}$ is a valid derivation ending in an application of rule $\mathcal{R}$ when all of these conditions hold:

- The derivation is obtained by substituting for the metavariables in $\mathcal{R}$. Substitution must respect the intent of the metavariables: only a well-formed

expression may be substituted for e , a value for v , a well-formed environment for ρ , and so on. While whatever is substituted must be well formed, it need not be completely specified: whatever is substituted for a metavariable may itself contain metavariables. For example, $\text{IF}(e_1, e_2, e_3)$ may be substituted for e .

- Rule $\mathcal{R}$ may include evaluation judgments above the line, as premises. After substitution, each of these premises must be justified by a derivation of its own.
- After substitution, every other premise in rule $\mathcal{R}$ must also be justified. Premises that aren't evaluation judgments are usually justified by set theory, arithmetic, or appeal to assumptions.

As an example, if n is a formal parameter, evaluating $(\text{set } n \ 0)$ sets n to zero and returns zero. I want to derive the judgment

$$\langle \text{SET}(n, \text{LITERAL}(0)), \xi, \phi, \rho \rangle \Downarrow \langle 0, \xi, \phi, \rho\{n \mapsto 0\} \rangle.$$

This goal judgment is derived by a derivation $\mathcal{D}$ that ends in rule **FORMALASSIGN**, which I reproduce here:

$$\frac{x \in \text{dom } \rho \quad \langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}{\langle \text{SET}(x, e), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho'\{x \mapsto v\} \rangle} \quad (\text{FORMALASSIGN})$$

To get the conclusion of **FORMALASSIGN** to match the goal, I substitute n for x , $\text{LITERAL}(0)$ for e , ξ for ξ' , 0 for v , and ρ for ρ' . With that substitution, my derivation in progress looks like this:

$$\frac{n \in \text{dom } \rho \quad \langle \text{LITERAL}(0), \xi, \phi, \rho \rangle \Downarrow \langle 0, \xi, \phi, \rho \rangle}{\langle \text{SET}(n, \text{LITERAL}(0)), \xi, \phi, \rho \rangle \Downarrow \langle 0, \xi, \phi, \rho\{n \mapsto 0\} \rangle}.$$

To complete the derivation, I derive judgment $\langle \text{LITERAL}(0), \xi, \phi, \rho \rangle \Downarrow \langle 0, \xi, \phi, \rho \rangle$, with a new derivation $\mathcal{D}_1$. Derivation $\mathcal{D}_1$ ends in rule **LITERAL**, and to construct $\mathcal{D}_1$, I substitute 0 for v in rule **LITERAL**. Rule **LITERAL** has no premises, so that substitution completes $\mathcal{D}_1$:

$$\mathcal{D}_1 = \frac{}{\langle \text{LITERAL}(0), \xi, \phi, \rho \rangle \Downarrow \langle 0, \xi, \phi, \rho \rangle}.$$

Plugging $\mathcal{D}_1$ into my previous work completes $\mathcal{D}$:

$$\mathcal{D} = \frac{n \in \text{dom } \rho \quad \overline{\langle \text{LITERAL}(0), \xi, \phi, \rho \rangle \Downarrow \langle 0, \xi, \phi, \rho \rangle}}{\langle \text{SET}(n, \text{LITERAL}(0)), \xi, \phi, \rho \rangle \Downarrow \langle 0, \xi, \phi, \rho\{n \mapsto 0\} \rangle}.$$

For derivation $\mathcal{D}$ to be valid, premise $n \in \text{dom } \rho$ must also be proved. It's true by assumption: n is a formal parameter, which is what $n \in \text{dom } \rho$ means.

How do we construct valid derivations?

A derivation is valid if we substitute consistently for each metavariable, replace every evaluation premise with a valid derivation, and can prove the other premises. But how do we know what to substitute? There's an algorithm, and it's the same algorithm that's used in `eval` to interpret code. That's no surprise—every derivation tree is intimately related to an application of `eval`. The derivation expresses, in a data structure, the steps that `eval` goes through to interpret the code.

To construct a derivation, we start with an initial state—the left-hand side of an evaluation judgment. We usually know some or all of the syntax, and we may know something about environments as well. The algorithm works like this:

1. Find a rule $\mathcal{R}$ whose *conclusion* matches the left-hand side we are interested in. By “matches,” I mean that the left-hand side we are interested can be obtained by substituting for metavariables in the conclusion of $\mathcal{R}$. For example, in the derivation for the SET expression above, only two rules can conclude in a SET expression: `FORMALASSIGN` and `GLOBALASSIGN`.

In the Impcore interpreter, the “find a rule” step corresponds to the main switch statement in function `eval` (chunk 48b), which identifies possible rules by looking at `e→alt`.

2. Substitute in $\mathcal{R}$ so that the left-hand side of the conclusion is equal to the initial state of interest. Now look at each of $\mathcal{R}$ ’s premises.

3. If a premise does not involve an evaluation judgment, see if it is provable. If so, continue building the derivation. If not, try some other rule.

In an interpreter, provability corresponds to a check on run-time data. Premises like $x \in \text{dom } \xi$ or $x \notin \text{dom } \rho$ are checked by calling `isvalbound`, and premises like $v_1 \neq 0$ or $v_1 = 0$ are checked by comparing a run-time value with 0.

4. If a premise *does* involve an evaluation judgment, recursively build a derivation of that judgment, starting with the right-hand side. In the Impcore interpreter, recursively building a derivation corresponds to a recursive call to `eval`.

This algorithm eventually dictates what needs to be substituted on the right-hand side of the evaluation judgment. An example appears in Figure 1.7, which constructs a derivation for `(set n (+ n 1))` in an environment in which `n` is 7. Figure 1.7 shows a sequence of snapshots; in each one, the partially constructed derivation is shown in black and the parts that are not yet constructed are shown in gray. The derivation begins with the expression `(set n (+ n 1))` and an environment ρ in which $\rho(n) = 8$. (Because ξ and ϕ do not take part and do not change, they are omitted to save space.) Each step fills something in; first, check $n \in \text{dom } \rho$; next, evaluate `(+ n 1)`, and so on. The sketch should suggest that the derivation, which is a data structure, is a spacelike representation of a timelike computation. This is true of every derivation.

What can we do with derivations?

If we know something about the environments, we can use derivation trees to answer questions about the evaluation of expressions and definitions. One example is the evaluation of the expression in Exercise 12 on page 77. This kind of application of the language semantics is called the *theory* of the language. But we can answer much more interesting questions if we *prove facts about derivations*. For example, in Impcore, the expression `(if x x 0)` is always equivalent to `x` (Exercise 13 on page 77). Because a computation is a sequence of events in time, proving facts about computations can be difficult. But if every terminating computation is described by a derivation, a derivation is just a data structure, and proving facts about data structures is much easier. Reasoning about derivations is called *metatheory*.

1.7.2 Proofs about derivations: Metatheory

If you already know that you want to study metatheory, you’ll soon need another book. This book only suggests what metatheory can do, so you can figure out if you want to study it more deeply.

$$\begin{array}{c}
\frac{n \in \text{dom } \rho}{\langle n, \rho \rangle \Downarrow \langle 7, \rho \rangle} \quad \frac{}{\langle 1, \rho \rangle \Downarrow \langle 1, \rho \rangle} \\
\frac{n \in \text{dom } \rho \quad \langle (+ \ n \ 1), \rho \rangle \Downarrow \langle 8, \rho \rangle}{\langle (\text{set } n \ (+ \ n \ 1)), \rho \rangle \Downarrow \langle 8, \rho\{n \mapsto 8\} \rangle}
\end{array}
\qquad
\begin{array}{c}
\frac{n \in \text{dom } \rho}{\langle n, \rho \rangle \Downarrow \langle 7, \rho \rangle} \quad \frac{}{\langle 1, \rho \rangle \Downarrow \langle 1, \rho \rangle} \\
\frac{n \in \text{dom } \rho \quad \langle (+ \ n \ 1), \rho \rangle \Downarrow \langle 8, \rho \rangle}{\langle (\text{set } n \ (+ \ n \ 1)), \rho \rangle \Downarrow \langle 8, \rho\{n \mapsto 8\} \rangle}
\end{array}$$

$$\begin{array}{c}
\frac{n \in \text{dom } \rho}{\langle n, \rho \rangle \Downarrow \langle 7, \rho \rangle} \quad \frac{}{\langle 1, \rho \rangle \Downarrow \langle 1, \rho \rangle} \\
\frac{n \in \text{dom } \rho \quad \langle (+ \ n \ 1), \rho \rangle \Downarrow \langle 8, \rho \rangle}{\langle (\text{set } n \ (+ \ n \ 1)), \rho \rangle \Downarrow \langle 8, \rho\{n \mapsto 8\} \rangle}
\end{array}
\qquad
\begin{array}{c}
\frac{n \in \text{dom } \rho}{\langle n, \rho \rangle \Downarrow \langle 7, \rho \rangle} \quad \frac{}{\langle 1, \rho \rangle \Downarrow \langle 1, \rho \rangle} \\
\frac{n \in \text{dom } \rho \quad \langle (+ \ n \ 1), \rho \rangle \Downarrow \langle 8, \rho \rangle}{\langle (\text{set } n \ (+ \ n \ 1)), \rho \rangle \Downarrow \langle 8, \rho\{n \mapsto 8\} \rangle}
\end{array}$$

$$\begin{array}{c}
\frac{n \in \text{dom } \rho}{\langle n, \rho \rangle \Downarrow \langle 7, \rho \rangle} \quad \frac{}{\langle 1, \rho \rangle \Downarrow \langle 1, \rho \rangle} \\
\frac{n \in \text{dom } \rho \quad \langle (+ \ n \ 1), \rho \rangle \Downarrow \langle 8, \rho \rangle}{\langle (\text{set } n \ (+ \ n \ 1)), \rho \rangle \Downarrow \langle 8, \rho\{n \mapsto 8\} \rangle}
\end{array}
\qquad
\begin{array}{c}
\frac{n \in \text{dom } \rho}{\langle n, \rho \rangle \Downarrow \langle 7, \rho \rangle} \quad \frac{}{\langle 1, \rho \rangle \Downarrow \langle 1, \rho \rangle} \\
\frac{n \in \text{dom } \rho \quad \langle (+ \ n \ 1), \rho \rangle \Downarrow \langle 8, \rho \rangle}{\langle (\text{set } n \ (+ \ n \ 1)), \rho \rangle \Downarrow \langle 8, \rho\{n \mapsto 8\} \rangle}
\end{array}$$

$$\begin{array}{c}
\frac{n \in \text{dom } \rho}{\langle n, \rho \rangle \Downarrow \langle 7, \rho \rangle} \quad \frac{}{\langle 1, \rho \rangle \Downarrow \langle 1, \rho \rangle} \\
\frac{n \in \text{dom } \rho \quad \langle (+ \ n \ 1), \rho \rangle \Downarrow \langle 8, \rho \rangle}{\langle (\text{set } n \ (+ \ n \ 1)), \rho \rangle \Downarrow \langle 8, \rho\{n \mapsto 8\} \rangle}
\end{array}$$

Figure 1.7: Construction of a derivation (omitting unchanging ξ and ϕ)

Metatheory can determine the validity of a claim like this: in any Impcore program, the expression $(+ \ x \ \theta)$ can be replaced by just x , and this replacement doesn't change any output of the program. This claim could be important to a compiler writer, who might use it to create an “optimization” that improves performance. If you're going to create an optimization, you must be certain that it doesn't change the meaning of any program. Certainty can be supplied by a metatheoretic proof.

What can we prove about a program that evaluates $(+ \ x \ \theta)$? If a derivation exists, it will contain a judgment of the form

$$\langle (+ \ x \ \theta), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle.$$

That judgment will be the root of a subderivation. That subderivation must apply a rule that permits the application of the $+$ primitive on the left-hand side of its conclusion. So just like the interpreter code in chunk 48b, a proof has to consider inference rules whose conclusions can contain `APPLY(+, ...)`.

Two such rules appear in Section 1.5.6: `APPLYUSER` and `APPLYADD`. And the claim is false! If $\phi(+)$ refers to a user-defined function, the `APPLYUSER` rule kicks in, and it is *not* safe to replace $(+ \ x \ \theta)$ with x . Here's a demonstration:

```

60. <terrifying transcript 60> ≡
    -> (define + (x y) y)           ; no sane person would do this
    -> (define addzero (x) (+ x 0))
    -> (addzero 99)
    0
    -> (define addzero2 (x) x)
    -> (addzero2 99)
    99

```

If a compiler writer wants to be able to replace an occurrence of $(+ \times \theta)$ with x , they will first have to prove that the environment ϕ in which $(+ \times \theta)$ is evaluated *never* binds $+$ to a user-defined function. (Compilers typically include lots of infrastructure for proving facts about environments, but such infrastructure is beyond the scope of this book.)

Proving facts about derivations is metatheory. Metatheory enables you to prove properties like these:

- Expression $(\text{if } x \times \theta)$ is equivalent to x (Exercise 13).
- If evaluation of a `while` loop terminates, its value is zero (Exercise 23).
- Evaluating an expression can't create a new variable (Exercise 24).
- In Impcore, evaluating an expression in the same context always produces the same result—which isn't true of languages that support parallel execution (Exercise 27).
- Impcore programs can be evaluated using a stack of mutable environments, as the implementation in Section 1.6 does (Exercise 29).

Results like these are useful, but one requires a long, detailed proof.

1.7.3 How to attempt a metatheoretic proof

A metatheoretic proof works by induction on the structure of valid derivations. A derivation is a tree, and a proof can assume that an induction hypothesis holds for any proper subtree. Or, if you prefer, a metatheorem can be proved by induction on the height of a derivation tree.

A valid derivation can end in any rule, so a proof by induction on a derivation's structure has a case for every rule. In a language as big as Impcore, that's a lot of cases. In this book, each case is presented in exactly the same way. As an example, here's a case from the proof that evaluating an expression doesn't create any new global variables. The theorem, which is also the induction hypothesis, says that if derivation $\mathcal{D}$ proves $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle$, then $\text{dom } \xi = \text{dom } \xi'$:

- When the last rule used in $\mathcal{D}$ is `FORMALASSIGN`, the derivation must have the following form:

$$\mathcal{D} = \frac{x \in \text{dom } \rho \quad \frac{\mathcal{D}_1}{\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}}{\langle \text{SET}(x, e_1), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \{x \mapsto v\} \rangle} \text{FORMALASSIGN}$$

The form of e is $\text{SET}(x, e_1)$. Our obligation is to prove that the induction hypothesis holds for the judgment below the line. We must therefore prove that $\text{dom } \xi = \text{dom } \xi'$. But because derivation $\mathcal{D}_1$ is smaller than derivation $\mathcal{D}$, we are permitted to assume the induction hypothesis, which tells us that $\text{dom } \xi = \text{dom } \xi'$. Our obligation is met.

This example, like all the cases in my metatheoretic proofs, uses the following template:

1. When the last rule used in $\mathcal{D}$ is `RULENAME`, and `RULENAME` has conclusion C and premises P_1 to P_n , the derivation must have the following form:

$$\mathcal{D} = \frac{P_1 \quad \cdots \quad P_n}{C} \text{RULENAME}$$

Commentary: The conclusion C is the evaluation judgment of which $\mathcal{D}$ is a proof. If any particular P_i is also an evaluation judgment, I write its derivation above it, as in

1

$$\mathcal{D} = \frac{\frac{\mathcal{D}_1}{P_1} \quad \dots \quad \frac{\mathcal{D}_i}{P_i} \quad \dots \quad \frac{\mathcal{D}_n}{P_n}}{C} \text{ RULENAME}$$

An imperative core

62

A premise like “ $x \notin \text{dom } \rho$ ” is not an evaluation judgment and is not supported by a subderivation.

2. The form of e is syntactic form, and whatever additional analysis goes with that syntactic form and with rule RULENAME.
3. Our obligation is to prove that the induction hypothesis holds for the judgment below the line. We must therefore prove *whatever it is*.
4. Identifying each premise P_i that is an evaluation judgment, because derivation $\mathcal{D}_i$ is strictly smaller than derivation $\mathcal{D}$, we are permitted to assume that the induction hypothesis applies to derivation $\mathcal{D}_i$. This assumption gives us *whatever it gives us*.
5. From the truth of premises P_1 to P_n , plus the information from the induction hypothesis, we show that *the induction hypothesis holds for the judgment below the line*.
6. Our obligation is met.

This template has served me well, but part of it may surprise you: it doesn’t distinguish between “base cases” and “inductive cases.” The distinction exists—a base case is one that has no evaluation judgments above the line—but in a programming-language proof, the distinction is not terribly useful. For example, base cases might or might not be easy, and they might or might not fail.

The template is instantiated for every case in a proof. As a demonstration, I instantiate the template to try to prove the metatheoretic conjecture,

If an expression e is evaluated successfully, then every variable in e is defined.

(This conjecture isn’t true, but that’s a good thing—we learn the most from the things we try to prove that aren’t so. To maximize your own learning, you might pause and think about which case or cases of the proof are going to fail.)

To begin, I state my conjecture formally. And that means I must formalize the idea of “every variable in e .” I use the function $\text{fv}(e)$, short for “free variables of e ,” which is defined in Figure 1.8 on the facing page. Figure 1.8 uses a simplified dialect of Impcore, which makes the metatheoretic proof a little easier:

- Simplified Impcore has no `begin` expression.
- In Simplified Impcore, every function application has exactly two arguments.

(You can work out how to eliminate `begin` in Exercise 14 on page 79.) Informally speaking, $\text{fv}(e)$ is the set of variables mentioned in e . A variable can be mentioned directly only in a `VAR` expression or in a `SET` expression. In other forms of expression, the free variables are the free variables of the subexpressions.

$$\begin{aligned}
\text{fv}(\text{LITERAL}(v)) &= \emptyset \\
\text{fv}(\text{VAR}(x)) &= \{x\} \\
\text{fv}(\text{SET}(x, e)) &= \{x\} \cup \text{fv}(e) \\
\text{fv}(\text{IF}(e_1, e_2, e_3)) &= \text{fv}(e_1) \cup \text{fv}(e_2) \cup \text{fv}(e_3) \\
\text{fv}(\text{WHILE}(e_1, e_2)) &= \text{fv}(e_1) \cup \text{fv}(e_2) \\
\text{fv}(\text{APPLY}(f, e_1, e_2)) &= \text{fv}(e_1) \cup \text{fv}(e_2)
\end{aligned}$$

Figure 1.8: Free variables of an expression in Simplified Impcore

Now I can state my conjecture precisely and formally: if $\mathcal{D}$ is a valid derivation of $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle$, then $\text{fv}(e) \subseteq \text{dom } \xi \cup \text{dom } \rho$. This conjecture is also my induction hypothesis.

I now instantiate the proof template for each rule of Simplified Impcore. (Rules are listed in Figure 1.11 on page 78.)

- When the last rule used in $\mathcal{D}$ is LITERAL, the derivation must have the following form:

$$\mathcal{D} = \frac{}{\langle \text{LITERAL}(v), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi, \phi, \rho \rangle} \text{LITERAL}$$

The form of e is $\text{LITERAL}(v)$. Our obligation is to prove that the induction hypothesis holds for the judgment below the line. We must therefore prove $\text{fv}(\text{LITERAL}(v)) \subseteq \text{dom } \xi \cup \text{dom } \rho$. According to the definition of fv in Figure 1.8, $\text{fv}(\text{LITERAL}(v)) = \emptyset$, and the empty set is a subset of any set. Our obligation is met.

- When the last rule used in $\mathcal{D}$ is FORMALVAR, the derivation must have the following form:

$$\mathcal{D} = \frac{x \in \text{dom } \rho}{\langle \text{VAR}(x), \xi, \phi, \rho \rangle \Downarrow \langle \rho(x), \xi, \phi, \rho \rangle} \text{FORMALVAR}$$

The form of e is $\text{VAR}(x)$. Our obligation is to prove that the induction hypothesis holds for the judgment below the line. We must therefore prove $\text{fv}(\text{VAR}(x)) \subseteq \text{dom } \xi \cup \text{dom } \rho$. According to the definition of fv in Figure 1.8, $\text{fv}(\text{VAR}(x)) = \{x\}$. And from the first and only premise of the derivation, we know that $x \in \text{dom } \rho$. Therefore

$$\text{fv}(\text{VAR}(x)) = \{x\} \subseteq \text{dom } \rho \subseteq (\text{dom } \rho \cup \text{dom } \xi).$$

Our obligation is met.

- When the last rule used in $\mathcal{D}$ is GLOBALVAR, the derivation must have the following form:

$$\mathcal{D} = \frac{x \notin \text{dom } \rho \quad x \in \text{dom } \xi}{\langle \text{VAR}(x), \xi, \phi, \rho \rangle \Downarrow \langle \xi(x), \xi, \phi, \rho \rangle} \text{GLOBALVAR}$$

The form of e is $\text{VAR}(x)$. Our obligation is to prove that the induction hypothesis holds for the judgment below the line. We must therefore prove $\text{fv}(\text{VAR}(x)) \subseteq \text{dom } \xi \cup \text{dom } \rho$. According to the definition of fv in Figure 1.8,

$\text{fv}(\text{VAR}(x)) = \{x\}$. And from the second premise of the derivation, we know that $x \in \text{dom } \xi$. Therefore

$$\text{fv}(\text{VAR}(x)) = \{x\} \subseteq \text{dom } \xi \subseteq (\text{dom } \rho \cup \text{dom } \xi).$$

Our obligation is met.

- When the last rule used in $\mathcal{D}$ is **FORMALASSIGN**, the derivation must have the following form:

$$\mathcal{D} = \frac{x \in \text{dom } \rho \quad \frac{\mathcal{D}_1}{\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle}}{\langle \text{SET}(x, e_1), \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \{x \mapsto v\} \rangle} \text{FORMALASSIGN}$$

The form of e is $\text{SET}(x, e_1)$. Our obligation is to prove that the induction hypothesis holds for the judgment below the line. We must therefore prove that $\text{fv}(\text{SET}(x, e_1)) \subseteq \text{dom } \xi \cup \text{dom } \rho$. According to the definition of fv in Figure 1.8, $\text{fv}(\text{SET}(x, e_1)) = \{x\} \cup \text{fv}(e_1)$. It therefore suffices to prove the following two inclusions:

- (a) $\{x\} \subseteq \text{dom } \xi \cup \text{dom } \rho$.
- (b) $\text{fv}(e_1) \subseteq \text{dom } \xi \cup \text{dom } \rho$.

From the first premise of the derivation, we know that $x \in \text{dom } \rho$, and as before, that implies (a). From the second premise of the derivation, we can apply the induction hypothesis to $\mathcal{D}_1$, which gives us (b). Our obligation is met.

- When the last rule used in $\mathcal{D}$ is **GLOBALASSIGN**, the reasoning is so similar to what we have already done that it's not worth repeating.
- When the last rule used in $\mathcal{D}$ is **IFTRUE**, the derivation must have the following form:

$$\mathcal{D} = \frac{\frac{\mathcal{D}_1}{\langle e_1, \xi, \phi, \rho \rangle \Downarrow \langle v_1, \xi', \phi, \rho' \rangle} \quad v_1 \neq 0 \quad \frac{\mathcal{D}_2}{\langle e_2, \xi', \phi, \rho' \rangle \Downarrow \langle v_2, \xi'', \phi, \rho'' \rangle}}{\langle \text{IF}(e_1, e_2, e_3), \xi, \phi, \rho \rangle \Downarrow \langle v_2, \xi'', \phi, \rho'' \rangle} \text{IFTRUE}$$

The form of e is $\text{IF}(e_1, e_2, e_3)$. Our obligation is to prove that the induction hypothesis holds for the judgment below the line. We must therefore prove $\text{fv}(\text{IF}(e_1, e_2, e_3)) \subseteq \text{dom } \xi \cup \text{dom } \rho$. According to the definition of fv in Figure 1.8, $\text{fv}(\text{IF}(e_1, e_2, e_3)) = \text{fv}(e_1) \cup \text{fv}(e_2) \cup \text{fv}(e_3)$. It therefore suffices to prove the following three inclusions:

- (a) $\text{fv}(e_1) \subseteq \text{dom } \xi \cup \text{dom } \rho$.
- (b) $\text{fv}(e_2) \subseteq \text{dom } \xi \cup \text{dom } \rho$.
- (c) $\text{fv}(e_3) \subseteq \text{dom } \xi \cup \text{dom } \rho$.

By applying the induction hypothesis to $\mathcal{D}_1$, we get (a). By applying the induction hypothesis to $\mathcal{D}_2$, we get (b). But there is no subderivation corresponding to the evaluation of e_3 . And in fact we cannot prove that $\text{fv}(e_3) \subseteq \text{dom } \xi \cup \text{dom } \rho$, because it isn't true! Expression e_3 is not evaluated, and its free variables might not be defined. Here's an example:

64. $\langle \text{transcript } 12a \rangle + \equiv$
 $\rightarrow (\text{if } 1 \ 7 \ \text{undefined})$
 7

<29

Our obligation cannot be met.

- I could continue with the other cases. The proof fails for IFFALSE as well as IFTRUE, but it succeeds for EMPTYBEGIN, BEGIN, and APPLYUSER. Analysis of rules WHILEITERATE and WHILEEND is left for Exercise 26.

The conjecture isn't actually a theorem, and in the process of working out a proof, I found a counterexample. To guarantee that every variable in an expression is defined, we would need something stronger than a successful evaluation. Like a type checker, for example (Chapter 6).

§1.7
Operational
semantics
revisited: Proofs

65

1.7.4 Why bother with semantics, proofs, theory, and metatheory?

What's up with all the Greek letters and horizontal lines? What's the point? Isn't it easier just to look at the code? No, because an operational semantics leaves out all sorts of "implementation details" that would otherwise impede our understanding of how a language works. For example, to a compiler writer, the representation of an environment is super important—where values are stored has a huge impact on the performance of programs. But if we just want to understand how programs behave, we don't care. And once you get used to the Greek letters and horizontal lines, you'll find them easier to read than code—*much* easier. The point of operational semantics is to combine precision and understanding. That's why when you find a new idea in a professional paper, the idea is usually nailed down using operational semantics. When you can read operational semantics, you'll be able to learn about new ideas for yourself, direct from the sources, instead of having to find somebody to explain them to you.

What about proof theory and metatheory? Theory involves making derivations—typically one derivation at a time. It can guide an implementor, because it tells them just what each construct is supposed to do. In principle, theory could also guide a programmer, who also needs to know what programs are supposed to do. But in practice, operational semantics works at too low a level. A programmer can more effectively use something like the *algebraic laws* in the next chapter. The programmer—or perhaps a specialist—uses operational semantics to show that the laws are sound, and after that, programming proceeds by appealing to the laws, not to the operational semantics directly.

So theory is good for building implementations and for establishing algebraic laws, both of which are useful for programmers. What is metatheory good for? Metatheory involves *reasoning about derivations*. In particular, metatheory can reveal universal truths about derivations, which correspond to facts about all programs in a given language. Such truths might interest implementors, programmers, or even policy makers. For example,

- If you're implementing C or Impcore, you can keep the local variables and formal parameters of *all* functions on a stack (Exercise 29). This stack is called the *call stack*.
- In Impcore, no function can change the value of a formal parameter (or local variable) belonging to any other function.
- In C, a function *can* change the value of a formal parameter (or local variable) belonging to another function, but only if at some point the *&* operator was applied to the parameter or variable in question—or if somebody has exploited "undefined behavior" with pointers.

- If an ordinary device driver fails, it can take down a whole operating-system kernel, resulting in a “blue screen of death.” But if a device driver is written in the special-purpose language Sing#, the worst it can do is take away its device—metatheory guarantees that the operating-system kernel and the other drivers are unaffected.

Serious metatheory is well worth learning; this book provides just a taste. Doing a few of the exercises can show you the difference between theory and metatheory, give you an idea of how a metatheoretic proof is structured, and give you an idea of what metatheory can do.

1.8 EXTENDING IMPCORE

Impcore is a “starter kit” for learning about abstract syntax, operational semantics, and interpreters. It’s not a useful programming language—useful languages offer more values than just machine integers. (New values are coming in Chapter 2.) But even with only integer values, Impcore can still be extended in two useful ways: with local variables and with looping constructs.

Any language that even pretends to be useful offers some species of local variables. Impcore can offer them too (Exercise 30 on page 86). Local variables can be used to define functions like the one shown in Figure 1.9 on the next page, which adds up the odd numbers from 1 to n .

Adding local variables requires you to *change* the abstract syntax of Userfun so that it includes not only a body and a list of formal parameters, but also a list of local variables. And to account for the semantics of local variables, you’ll need to change the evaluator. But other kinds of extensions, including new looping constructs, can be implemented *without* touching the abstract syntax or the evaluator. You add only concrete syntax, which is implemented in terms of the abstract syntax you have already. This kind of new concrete syntax is called *syntactic sugar*.

As examples of syntactic sugar, I suggest several new ways to write loops. Let’s begin with an ordinary while loop, like this one:

```
(while (<= i n)
  (begin
    (set sum (+ sum i))
    (set i (+ i 2))))
```

The `begin` might seem like a lot of syntactic overhead. I imagine a new syntactic form, which I’ll call `while*`, in which the condition is still a single expression, but the body is a *sequence* of expressions. Now the `begin` is no longer necessary:

```
(while* (<= i n)
  (set sum (+ sum i))
  (set i (+ i 2)))
```

The `while*` loop can be defined as syntactic sugar:

$$(\text{while* } \text{condition } e_1 \cdots e_n) \triangleq (\text{while } \text{condition } (\text{begin } e_1 \cdots e_n))$$

As another example, C’s `do...while` loop executes the body first, then the condition. In Impcore, we might define a `do-while` as syntactic sugar:

$$(\text{do-while } \text{body } \text{condition}) \triangleq (\text{begin } \text{body } (\text{while } \text{condition } \text{body}))$$

Finally, C’s complicated four-part `for` loop can also be defined as syntactic sugar:

$$(\text{for } \text{pre } \text{test } \text{post } \text{body}) \triangleq (\text{begin } \text{pre } (\text{while } \text{test } (\text{begin } \text{body } \text{post})))$$

```

67. <answer transcript 67>≡
-> (define add-odds-to (n)
    [locals i sum]
    (begin
      (set i 1)
      (set sum 0)
      (while (<= i n)
        (begin
          (set sum (+ sum i))
          (set i (+ i 2))))
      sum))
-> (add-odds-to 3)
4
-> (add-odds-to 5)
9
-> (add-odds-to 7)
16

```

Figure 1.9: Programming with local variables

All these alternatives can be implemented just by modifying Impcore’s parser (Exercise 34, page 87). An example can be found in the Supplement (Section G.7, page S209).

1.9 SUMMARY

Impcore is a toy, but its simple, imperative control constructs—procedure definitions, conditionals, and loops—model the structure of all procedural languages. These constructs are also found in many other languages, including some that describe themselves as “object-oriented,” “scripting,” or even “functional.” More important, Impcore serves as a tiny, familiar medium with which to introduce two foundational ideas: abstract syntax and operational semantics. Finally, Impcore introduces the most distinctive feature of this book: the definitional interpreter.

1.9.1 Key words and phrases

ABSTRACT MACHINE An abstraction used to evaluate programs in **OPERATIONAL SEMANTICS**. An abstract machine has a state formed from mathematical objects; typical objects include **ENVIRONMENTS**, **ABSTRACT SYNTAX**, values, and *stores* (Chapter 2). The state of an abstract machine undergoes transitions that are described by an **OPERATIONAL SEMANTICS**.

ABSTRACT SYNTAX The underlying *tree* structure of a program’s source code. It is “abstract” in part because it abstracts away from such details as whether source code is written using round brackets and square brackets or whether it is written using semicolons and curly braces. Abstract syntax expresses the important truth about a program. Compare it with **CONCRETE SYNTAX**.

BASIS A *basis* comprises all the information available about a *particular* set of names. In Impcore, a basis is a pair of environments $\langle \phi, \xi \rangle$. A basis provides the context used to evaluate a definition, and evaluating a definition typically extends or alters the current basis. The basis available at startup is called the **INITIAL BASIS**.

What is syntactic sugar and who benefits?

Syntactic sugar is a means of defining syntax in terms of other syntax, without any operational semantics. Some examples appear in the text: C's `do...while` and C's `for` loop can be defined as syntactic sugar for various combinations of `while` and `begin` (page 66). Such syntactic sugar can benefit programmers, implementors, designers, theorists, and other tool builders.

Programmers benefit most from syntactic sugar when they don't know it's there. Syntactic sugar is defined by translation, which is not easy to think about—if every time you want to use a `do-while` you first have to mentally translate it into something else, that's not an aid; it's a stumbling block.

Implementors can benefit from syntactic sugar. If you implement a *desugaring* transformation on your syntax, then without any other change to your compiler or interpreter, you have a new language feature (Exercise 34). But as soon as your implementation gets serious—say you want to check types, as in Chapter 6, or you want to provide source-level-debugging—the syntactic sugar is not so useful, because you need to report errors or state in terms of the syntax the user wrote originally, not the desugared form.

Language designers and theorists benefit the most from syntactic sugar. For example, let's say you've completed Exercise 29: you've proven that Impcore can be evaluated on a stack. Now you want to add `do-while`, `for`, `while*`, or some other shiny new syntactic form. If the new form is just sugar—that is, if it is defined by translation into the original syntax, which you used in your proof—then you know the new, extended Impcore can *still* be evaluated on a stack. You don't have to consider any new cases in your proof, and you don't have to revisit any cases that you've already proven. This scenario describes a very effective use of syntactic sugar: a careful language designer benefits from small language, which is easy to prove things about, but the users benefit from a larger language, which is more attractive and makes it easier to say things idiomatically. Using syntactic sugar, a designer can have both.

I have assumed that new syntactic sugar can be created only by a language designer or implementor. This assumption holds for most languages, including C and Impcore. the vast majority of other languages. But using Lisp, Scheme, and related languages, new syntactic sugar can be created by ordinary programmers. This capability gives programmers many of the same powers as language designers (Section 2.14.4, page 171).

BIG-STEP SEMANTICS A species of OPERATIONAL SEMANTICS in which each JUDGMENT FORM expresses, in one step, the evaluation of syntax to produce a result. For example, a judgment $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi, \phi, \rho \rangle$ shows how an expression is evaluated to produce a value, or a judgment $\langle d, \xi, \phi \rangle \rightarrow \langle \xi', \phi' \rangle$ shows how a definition is evaluated to produce a new ENVIRONMENT. Big-step semantics is well aligned with the way we think about programs. Compare it with SMALL-STEP SEMANTICS.

COMPILER A language implementation that works by translating syntax into machine code. The machine code may be for a real hardware machine made by a manufacturer like Intel or ARM, in which case the compiler is called a *native-code* compiler. Or the machine code may be for a *virtual machine* like the Java Virtual Machine or the Squeak virtual machine.

CONCRETE SYNTAX The means by which a program's source code is written, as a *sequence* of characters or tokens. Concrete syntax specifies such details as what shape of brackets to use and where to use commas or semicolons. Concrete syntax is what we use when presenting a program to a computer or talking about programs with other people. Compare it with **ABSTRACT SYNTAX**.

DEFINITIONAL INTERPRETER A language implementation that is intended to implement the language's theory "directly," or that is otherwise intended to illustrate a language's theory. Almost sure to include a **PARSER** and **EVALUATOR**. All the interpreters in this book are definitional.

ENVIRONMENT An mapping that stores information about names. For example, in **Impcore**, the environment ξ stores the value of each global variable. In old-school **COMPILERS**, an environment may be referred to as a **SYMBOL TABLE**.

EVALUATION What a pointy-headed theorist talks about instead of "running code."

EVALUATOR A part of a language implementation that evaluates code directly. In this book, the evaluators evaluate **ABSTRACT SYNTAX**, but an evaluator may also evaluate intermediate code, virtual-machine code, or even machine code.

EXPRESSION-ORIENTED LANGUAGE A programming language in which conditional constructs, control-flow constructs, and assignments, like **if**, **while**, **begin**, and **set**, are expressions, not statements—and evaluating each produces a value. Not usually associated with **PROCEDURAL PROGRAMMING**.

GRAMMAR A set of formal rules that enumerates all the **SYNTACTIC FORMS** in each **SYNTACTIC CATEGORY**. A grammar produces the set of all programs that are grammatically well formed. A grammar can be designed to support a simple decision procedure that tells if a particular utterance was produced by the grammar, and if so, how. Such a decision procedure is embodied in a **PARSER**.

INITIAL BASIS The **BASIS** used when first evaluating a user's code. The initial basis contains all the **PRIMITIVE FUNCTIONS** and **PREDEFINED FUNCTIONS**.

JUDGMENT A claim in a formal system of proof. In programming languages, a judgment often describes the evaluation of some syntactic form; for example, $\langle (+\ 2\ 2), \xi, \phi, \rho \rangle \Downarrow \langle 4, \xi, \phi, \rho \rangle$ describes the evaluation of a function application. (Judgments about types are introduced in Chapter 6.)

JUDGMENT FORM A template for creating **JUDGMENTS**. For example, the form of the evaluation judgment for **Impcore** is $\langle e, \xi, \phi, \rho \rangle \Downarrow \langle v, \xi', \phi, \rho' \rangle$. A judgment form is transformed into to a **JUDGMENT** by substituting **ABSTRACT SYNTAX**, values, **ENVIRONMENTS**, or other entities for its **METAVARIABLES**.

METATHEORETIC PROOF A proof of a fact that is true of all valid derivations. Normally proceeds by **STRUCTURAL INDUCTION** on derivations.

METATHEORY Theorems about proofs. More broadly, mathematical tools for showing properties that are true for the execution of any program. Example properties might be that evaluating an expression never introduces a new global variable, or that **Impcore** can be evaluated on a stack, or that a secure language does not leak information. Compare it with **THEORY**.

METAVARIABLE A variable that stands for something used in a JUDGMENT FORM or elsewhere in a THEORY. Metavariables used in this book include

e for an expression
 d for a definition
 x for a *program* variable
 v for a value
 ρ for an ENVIRONMENT
 σ for a *store* (Chapter 2)

The name of a metavariable tells a reader what kind of thing it stands for, but unfortunately, no two authors agree on names. Many authors, like Harper (2012), use a mix of Greek and Roman letters, but both Pierce (2002) and Cardelli (1989) use only Roman letters.

NATURAL DEDUCTION A style of proof which is expressed using INFERENCE RULES with JUDGMENTS in the premises and conclusion. (In mathematical logic, these judgments would be called “propositions.”) This is the style of our BIG-STEP SEMANTICS; it supposedly accords with a “natural” way of reasoning. Natural deduction is associated with the mathematician Gerhard Gentzen.

OPERATIONAL SEMANTICS A precise way of describing the evaluation of a program. Usually comprises PROOF RULES for JUDGMENTS about program evaluation. The operational semantics gives enough information to write an EVALUATOR for a language. Varieties in this book include BIG-STEP SEMANTICS and SMALL-STEP SEMANTICS.

PARSER The part of a language’s implementation that translates CONCRETE SYNTAX to ABSTRACT SYNTAX. (A parser may also translate concrete syntax directly to intermediate code.)

PARSING The process of transforming CONCRETE SYNTAX into ABSTRACT SYNTAX. More generally, the process of recognizing concrete syntax. (Some compilers and interpreters skip abstract syntax and instead generate code directly in the PARSE.)

PREDEFINED FUNCTION A function that is available to every program written in a language, because by the time a user’s code is examined, the function’s definition has already been evaluated. Like the `mod` function in Impcore. A predefined function is not part of its language; it can be defined using PRIMITIVE FUNCTIONS.

PRIMITIVE FUNCTION A function that is built into a language or its implementation, like the `+` function in Impcore. A primitive function typically cannot be defined using the other parts of its language, so it is considered a sort of a part of its language.

PROCEDURAL PROGRAMMING A style in which programs are built by composing side-effecting SYNTACTIC FORMS in sequence. The forms typically produce no values and are called *commands* or *statements*. They are usually grouped into *procedures*. Procedural programs operate primarily on mutable data, and they tend to manipulate one machine word at a time. Impcore’s only data type, the machine integer, does fit in one word, but because it is immutable, Impcore is not a very good example of a procedural programming language.

READ-EVAL-PRINT LOOP An interactive mechanism by which a programmer can run code. A definition or an extended definition is first *read* from standard

input; then it is evaluated and its result is printed. And then the implementation loops, waiting for the next definition. A read-eval-print loop enables a programmer to easily use or explore any function or variable. Alternatives that don't support interactive exploration include command-line batch development, where a programmer uses operating-system commands to compile and run code, and app development, where code is developed and packaged on one platform and then shipped to run on another platform.

SMALL-STEP SEMANTICS A species of **OPERATIONAL SEMANTICS** in which each **JUDGMENT FORM** expresses the smallest possible increment of computation. Such a semantics exposes the intermediate steps of the computation. An example appears in Chapter 3. Small-step semantics can express more kinds of program behaviors than **BIG-STEP SEMANTICS**, and the **METATHEORETIC** proof techniques used with small-step semantics tend to be simpler.

SYMBOL TABLE A compiler writer's word for **ENVIRONMENT**.

SYNTACTIC CATEGORY A group of **SYNTACTIC FORMS** that are *grammatically* interchangeable. Examples of common syntactic categories include expressions, definitions, statements, and types.

SYNTACTIC FORM A template for creating a phrase in a programming language, like `(set  $x$   $e$ )`.

SYNTACTIC SUGAR A means of extending **CONCRETE SYNTAX** without changing **ABSTRACT SYNTAX**. Syntactic sugar is concrete syntax that is translated into existing abstract syntax. Syntactic sugar can appear in a program, but it doesn't make it to a theory or an **EVALUATOR**. Syntactic sugar can be used to add new loop forms to **Impcore**, for example (Section 1.8, page 66).

THEORY Theorems about programs. More broadly, mathematical tools that specify meaning and behavior of programs. Theory can be used to prove facts about individual programs and to specify an **EVALUATOR** or type checker. In this book, the theory of a language is its operational semantics plus, in Chapters 6 to 9, its type system. A language's theory may be used to create a **DEFINITIONAL INTERPRETER**. Compare it with **METATHEORY**.

1.9.2 Further reading

Literate programming was invented by Knuth (1984); Noweb is described by Ramsey (1994).

The term “basis” is taken from the Definition of Standard ML (Milner et al. 1997), where it refers to a collection of environments binding not only values but types, “signatures,” and other entities.

Operational semantics got a big boost from Plotkin (1981), who describes a style of operational semantics that is better suited to proving properties of programs than the natural-deduction style that we use. The natural-deduction style was introduced by Kahn (1987); it is better suited to specifying evaluators.

The notations used to describe grammars and semantics aren't standardized: the ideas are universal, but details vary. Common variations are ably described by Steele (2017), who also explains why the variations are mostly harmless.

The first extensible printer for C programs that I know of was created by Ken Thompson; his implementation appeared in Ninth Edition Research Unix. Another implementation can be found in Chapter 14 of Hanson (1996).